



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y
Tecnología

Máster Universitario en Inteligencia artificial

Desarrollo de un sistema local de vigilancia doméstica con modelos de visión artificial

Trabajo fin de estudio presentado por:	Javier Helguera López
Tipo de trabajo:	Desarrollo de Software
Director/a:	Ainhoa García Sánchez
Fecha:	Febrero 2026

Resumen

Este Trabajo Fin de Máster presenta el desarrollo de un sistema de videovigilancia doméstica inteligente diseñado para funcionar de manera completamente local, con el objetivo de mejorar la seguridad sin comprometer la privacidad del usuario. Frente a las soluciones comerciales basadas en suscripción y procesamiento en la nube, se propone una alternativa autónoma capaz de analizar vídeo en tiempo real mediante técnicas de visión por computador ejecutadas en hardware de bajos recursos.

La metodología seguida se basó en un enfoque ágil inspirado en Scrum, estructurando el proyecto en fases iterativas que abarcaron el análisis del problema, la selección de modelos eficientes, el diseño modular de la arquitectura y la implementación progresiva del sistema. La solución integra captura de vídeo mediante cámaras IP a través del protocolo RTSP, detección automática de personas usando el modelo ligero MobileNet-SSD y un dashboard web desarrollado con FastAPI para la gestión de cámaras, alertas y eventos. Adicionalmente, se incorporó un sistema de notificaciones mediante Telegram como canal externo controlado.

Los resultados experimentales muestran una precisión elevada en escenarios domésticos reales, con una reducción significativa de falsas alarmas respecto a sistemas tradicionales basados únicamente en movimiento. Las pruebas en una Raspberry Pi 5 confirman la viabilidad computacional del sistema para un número reducido de cámaras simultáneas.

El proyecto demuestra que es posible implementar una solución de videovigilancia inteligente, eficiente y respetuosa con la privacidad mediante procesamiento local y modelos optimizados, ofreciendo una base sólida para futuras ampliaciones.

Palabras clave: Videovigilancia, visión por computador, procesamiento local, privacidad, Edge AI

Abstract

This Master's Thesis presents the development of an intelligent home video surveillance system designed to operate entirely locally, with the aim of improving household security without compromising user privacy. Unlike commercial solutions that rely on subscriptions and cloud-based processing, this project proposes an autonomous alternative capable of performing real-time video analysis through computer vision techniques deployed on low-resource hardware.

The methodology followed an agile approach inspired by the Scrum framework, organizing the work into iterative phases covering problem analysis, selection of efficient deep learning models, modular system design, and progressive implementation. The solution integrates video capture from IP cameras using the RTSP protocol, automatic person detection through the lightweight MobileNet-SSD model, and a web-based dashboard developed with FastAPI for camera management, event monitoring, and alert visualization. Additionally, a controlled external notification mechanism was implemented through Telegram, enabling the user to receive intrusion alerts without exposing the system publicly.

Experimental results demonstrate high detection accuracy in realistic domestic scenarios, with a significant reduction of false alarms compared to traditional motion-based surveillance systems. Deployment tests on a Raspberry Pi 5 confirm the computational feasibility of the approach for a limited number of simultaneous camera streams.

The project shows that it is possible to implement an efficient, privacy-preserving smart surveillance solution using local processing and optimized computer vision models, providing a solid foundation for future enhancements and extensions.

Keywords: surveillance, self-hosted, computer vision, privacy, Edge AI

Índice de contenidos

1.	Introducción	1
1.1.	Motivación	1
1.2.	Planteamiento del trabajo	2
1.3.	Estructura del documento	3
2.	Contexto y estado del arte	4
2.1.	Contexto e introducción	4
2.2.	Base teórica	4
2.2.1.	Historia de la Inteligencia Artificial	5
2.2.2.	Visión por computador	7
2.2.3.	Aprendizaje automático y redes neuronales profundas	8
2.2.4.	Protocolo RTSP	10
2.2.5.	Procesamiento local y eficiencia computacional	12
2.3.	Aplicaciones relacionadas	13
2.3.1.	Sistemas de empresas con suscripción	13
2.3.2.	Soluciones privadas, pero sin suscripción	14
2.3.3.	Blue Iris Software	15
2.4.	Proyectos relacionados	16
2.4.1.	Video Surveillance System Using IP Camera for Target Person Detection	16
2.5.	Conclusiones	16
3.	Objetivos concretos y metodología de trabajo	18
3.1.	Objetivo general	18
3.2.	Objetivos específicos	18
3.3.	Metodología del trabajo	19
3.3.1.	Fase 1: Análisis del problema y definición de requisitos	19

3.3.2.	Fase 2: Investigación y selección de modelos de IA	19
3.3.3.	Fase 3: Diseño incremental de la arquitectura del sistema	20
3.3.4.	Fase 4: Implementación iterativa mediante sprints.....	20
3.3.5.	Fase 5: Evaluación experimental y análisis de métricas.....	20
3.3.6.	Fase 6: Análisis final y documentación del proyecto	21
4.	Identificación de requisitos	22
4.1.	Requisitos funcionales.....	22
4.2.	Requisitos no funcionales.....	22
5.	Descripción de la herramienta de software desarrollada.....	24
5.1.	Entorno de desarrollo.....	24
5.1.1.	Ordenador de desarrollo	24
5.1.2.	Máquina virtual servidor (Ubuntu Server 24.04.3)	25
5.1.3.	Raspberry Pi 5	25
5.1.4.	Cámaras de video	26
5.2.	Arquitectura.....	28
5.3.	Captura de vídeo	30
5.3.1.	Servidor RTSP MediaMTX	30
5.3.2.	FFmpeg para manejar el flujo de vídeo	31
5.3.3.	Conectar FFMpeg y MediaMTX	32
5.3.4.	Visualizar el flujo de vídeo RTSP con Python.....	37
5.4.	Detección de personas mediante modelos de aprendizaje profundo	39
5.4.1.	Selección del modelo: MobileNet-SSD	39
5.4.2.	Uso de modelos preentrenados y transferencia de conocimiento.....	40
5.4.3.	Implementación del módulo de detección de personas.....	40
5.5.	Creación del dashboard.....	45

5.5.1.	Funcionalidad del dashboard	45
5.5.2.	Elección del Backend y Frontend: FastAPI.....	45
5.5.3.	Endpoints del Backend	46
5.5.4.	Integración de los módulos de visión artificial.....	48
5.5.5.	Persistencia de datos.....	50
5.5.6.	Frontend	54
5.6.	Bot de Telegram con acceso limitado desde el exterior	62
5.6.1.	Creación del bot.....	62
5.6.2.	Configuración y puesta en marcha	65
5.7.	Contenedor Docker listo para el despliegue	70
5.7.1.	Dockerfile.....	71
5.7.2.	Docker Compose.....	71
5.8.	Guía de usuario.....	73
6.	Evaluación	74
6.1.	Metodología empírica	74
6.2.	Métricas empleadas	75
6.3.	Resultados de eficacia de detección	76
6.4.	Rendimiento computacional	80
7.	Conclusiones y trabajo futuro	82
7.1.	Conclusiones.....	82
7.2.	Líneas de trabajo futuro	83
	Referencias bibliográficas.....	84
Anexo A.	Código en Python para capturar el vídeo del protocolo RTSP	88
Anexo B.	Dockerfile de la aplicación completa.....	90

.....	90
Anexo C. docker-compose de la aplicación completa	92
Anexo D. Repositorio GitHub	92

Índice de ilustraciones

Ilustración 1. Máquina virtual Ubuntu corriendo.....	25
Ilustración 2. Instalación de la cámara Tapo usada para pruebas en entorno real	27
Ilustración 3. Captura de una cámara Tapo emitiendo video por RTSP.....	27
Ilustración 4. Esquema de la arquitectura a alto nivel de la solución.....	29
Ilustración 5. Lista de cámaras disponibles en /dev/video*	32
Ilustración 6. Servidor MediamMTX corriendo	33
Ilustración 7. FFmpeg enviando el flujo de video al servidor RTSP MediaMTX	34
Ilustración 8. Ejemplo del flujo de video de la webcam USB mediante el protocolo RTSP	35
Ilustración 9. Estructura de ficheros de configuración para múltiples servidores RTSP.....	35
Ilustración 10. Ejemplo de dos cámaras emitiendo flujo de vídeo por RTSP simultáneamente	36
Ilustración 11. Localización de los ficheros necesarios para MobileNetSSD en el proyecto ...	41
Ilustración 12. Ejemplo de detección de una persona en condiciones de poca luz.....	44
Ilustración 13. Estructura del proyecto con el módulo independiente de detección de personas	48
Ilustración 14. Modelos de la base de datos	51
Ilustración 15. Ventana de login.....	56
Ilustración 16. Ventana de registro de usuario	57
Ilustración 17. Ventana principal del dashboard.....	58
Ilustración 18. Ventana de gestión de cámaras del sistema	59
Ilustración 19. Ventana para añadir una cámara al sistema	60
Ilustración 20. Ventana de gestión de alertas del sistema.....	61
Ilustración 21. Ventana de configuración del bot de Telegram	65
Ilustración 22. Conversación con BotFather en Telegram para la creación del bot	66

Ilustración 23. Envío del primer mensaje al nuevo bot.....	67
Ilustración 24. Información disponible en la URL getUpdates	68
Ilustración 25. Ventana de configuración del bot desde el dashboard	68
Ilustración 26. Ejemplo de una alerta de intrusión recibido en el bot.....	69
Ilustración 27. Ejemplo de respuesta del bot al comando /status.....	70
Ilustración 28. Ejemplo de detección de un intruso de espaldas con baja iluminación.	79
Ilustración 29. Ejemplo de detección de un intruso de frente a la cámara con iluminación diurna.....	80

Índice de tablas

Tabla 1. Comandos del protocolo RTSP.....	11
Tabla 2. Características del ordenador de desarrollo	24
Tabla 3. Métricas obtenidas en las pruebas de detección (TP, FP, FN y precisión) en función de la distancia (1 m, 5 m, 10 m) y el umbral de confianza configurado.	77
Tabla 4. Resultados medios de Precision, Recall y F1-Score en las pruebas de movimiento frontal hacia la cámara.	78
Tabla 5. Resultados medios de Precision, Recall y F1-Score en las pruebas de movimiento de espaldas.	78
Tabla 6. Resultados medios de Precision, Recall y F1-Score en las pruebas de presencia parcial del cuerpo.....	78
Tabla 7. Resultados medios de Precision, Recall y F1-Score en las pruebas de luz diurna.....	78
Tabla 8. Resultados medios de Precision, Recall y F1-Score en las pruebas de luz nocturna..	79
Tabla 9. Métricas de rendimiento obtenidas durante la ejecución simultánea de múltiples cámaras en Raspberry Pi 5 (8 GB), incluyendo FPS medio, latencia y uso de CPU/RAM.....	81

1. Introducción

La seguridad en el ámbito doméstico se ha convertido en una preocupación creciente debido al aumento del acceso a tecnologías de monitorización y a la necesidad de proteger espacios privados de manera eficaz. En este contexto, los sistemas de videovigilancia han evolucionado desde simples cámaras de grabación hacia soluciones inteligentes capaces de analizar automáticamente el contenido visual capturado. Esta transformación ha sido posible gracias al avance de la Inteligencia Artificial, especialmente en el campo de la visión por computador, que permite detectar objetos, reconocer rostros o identificar situaciones relevantes en tiempo real.

No obstante, la adopción masiva de estos sistemas también ha evidenciado importantes limitaciones. Muchas plataformas actuales dependen de servicios externos para procesar imágenes y almacenar grabaciones, lo que introduce riesgos relacionados con la privacidad, la seguridad de los datos personales y la dependencia de infraestructuras remotas. Paralelamente, el uso de modelos avanzados de IA suele requerir recursos computacionales elevados, dificultando su implementación en dispositivos accesibles o de bajo consumo.

Ante esta situación, este proyecto plantea el desarrollo de un sistema de videovigilancia inteligente que funcione de forma completamente local, combinando privacidad y eficiencia. Se propone una solución capaz de integrar técnicas modernas de detección de personas sin necesidad de enviar información a la nube, y que además pueda ejecutarse en hardware de recursos limitados, ofreciendo así una alternativa más segura, accesible y autónoma.

1.1.MOTIVACIÓN

En los últimos años, los sistemas de videovigilancia doméstica han adquirido una presencia cada vez mayor en hogares de todo el mundo. La facilidad de acceso a cámaras IP y dispositivos inteligentes ha impulsado su adopción como medida de seguridad habitual. Sin embargo, la mayoría de soluciones comerciales actuales presentan limitaciones importantes que afectan directamente a la experiencia del usuario y a la confianza depositada en estos sistemas.

En muchos casos, las herramientas convencionales se basan en detección de movimiento simple, generando un número elevado de falsas alarmas provocadas por factores cotidianos

como cambios de iluminación, sombras o animales. Esta falta de precisión disminuye la utilidad real del sistema, ya que las alertas frecuentes terminan perdiendo relevancia.

Además, la incorporación de técnicas avanzadas como el reconocimiento facial o el análisis inteligente de vídeo suele estar ligada al procesamiento en la nube. Esto implica que grabaciones e imágenes capturadas en espacios privados deben enviarse a servidores externos, lo que plantea preocupaciones significativas en términos de privacidad y control sobre los datos, junto con costes adicionales derivados de suscripciones.

Por todo ello, surge la necesidad de desarrollar alternativas capaces de aprovechar el potencial de la Inteligencia Artificial de manera local, garantizando la confidencialidad del usuario y reduciendo la dependencia de servicios externos. Este proyecto responde a dicha necesidad proponiendo un sistema de vigilancia inteligente centrado en la privacidad y optimizado para funcionar en dispositivos de bajos recursos.

1.2. PLANTEAMIENTO DEL TRABAJO

El punto de partida de este Trabajo Fin de Máster es la necesidad de contar con sistemas de videovigilancia doméstica más inteligentes, seguros y respetuosos con la privacidad. Las soluciones tradicionales suelen limitarse a la detección de movimiento, lo que provoca numerosas falsas alarmas, mientras que las alternativas más avanzadas dependen habitualmente de servicios en la nube, implicando el envío y almacenamiento de datos sensibles en infraestructuras externas.

Ante este problema, se plantea como finalidad principal el desarrollo de un sistema de videovigilancia que permita analizar vídeo en tiempo real de manera local, reduciendo la dependencia de terceros y garantizando un mayor control sobre la información capturada en el entorno doméstico. El objetivo general del trabajo es proponer una solución capaz de detectar la presencia de personas y distinguir entre usuarios autorizados e intrusos mediante técnicas de visión por computador basadas en Inteligencia Artificial.

La propuesta se enmarca dentro del ámbito de la ingeniería del software, la visión por computador y la privacidad de datos, ya que requiere el diseño de una arquitectura modular, el procesamiento eficiente de datos sensibles y la implementación de mecanismos orientados a proteger la privacidad del usuario. Asimismo, se desarrolla una aplicación web que permita gestionar el sistema, visualizar eventos y recibir alertas ante posibles amenazas.

1.3. ESTRUCTURA DEL DOCUMENTO

El documento se ha dividido en siete partes principales:

1. Introducción: se presenta el proyecto junto con la motivación y el planteamiento del trabajo.
2. Contexto y estado del arte: sirve como punto de partida del proyecto de investigación previo al desarrollo de la solución software. Se analizan las opciones similares ya existentes y se comparan con la propuesta para justificar por qué la solución aporta valor.
3. Objetivos y metodología de trabajo: se establecen los objetivos generales y específicos del proyecto, así como la metodología seguida y las fases aplicadas durante el desarrollo.
4. Identificación de requisitos: se recogen y organizan los requisitos funcionales y no funcionales necesarios para el diseño del sistema, incluyendo aspectos de rendimiento, seguridad y privacidad.
5. Descripción de la herramienta de software desarrollada: se detalla la arquitectura e implementación del sistema, incluyendo la integración de modelos de visión por computador, el procesamiento local y la interfaz de web de gestión.
6. Evaluación: se analizan los resultados obtenidos mediante pruebas de funcionamiento y rendimiento, valorando la eficacia del sistema en la detección de personas e intrusos en entornos de bajos recursos.
7. Conclusiones y trabajo futuro: se presentan las conclusiones finales alcanzadas y se proponen posibles líneas de mejora y ampliación del sistema para desarrollos posteriores.

2. CONTEXTO Y ESTADO DEL ARTE

2.1.CONTEXTO E INTRODUCCIÓN

En esta sección se analiza el contexto general en el que se enmarca el proyecto, así como los antecedentes necesarios para comprender su desarrollo. Con este objetivo, el capítulo se ha estructurado en dos bloques principales, combinando una base teórica con una revisión de soluciones similares existentes tanto en el ámbito comercial como en el científico.

El primer apartado está dedicado a los fundamentos conceptuales. En él se introduce progresivamente el campo de la Inteligencia Artificial, acotando el enfoque hasta llegar a la visión por computador, una de sus ramas más relevantes y la base tecnológica principal sobre la que se construye este proyecto. Se considera esencial exponer la teoría que sustenta los modelos empleados, ya que permite entender las razones detrás de su selección y su adecuación al problema planteado.

El segundo apartado se centra en el análisis de aplicaciones relacionadas. Se revisan distintas soluciones actuales de videovigilancia doméstica disponibles en el mercado, destacando sus ventajas y limitaciones, especialmente en aspectos como privacidad, accesibilidad y dependencia de servicios externos. Asimismo, se incluyen referencias a trabajos científicos recientes que respaldan la necesidad del enfoque propuesto y justifican la aportación y novedad de la solución desarrollada.

2.2.BASE TEÓRICA

En esta sección se presenta la base teórica necesaria para comprender las técnicas y tecnologías empleadas en el desarrollo del sistema de vigilancia doméstica propuesto. El proyecto se apoya principalmente en conceptos de visión por computador y aprendizaje automático, con especial énfasis en la detección de personas a partir de secuencias de vídeo.

El objetivo de este apartado es proporcionar los fundamentos que permitan entender el funcionamiento de los algoritmos utilizados, así como justificar su elección dentro del contexto del problema abordado.

2.2.1. Historia de la Inteligencia Artificial

La Inteligencia Artificial es una de las disciplinas más relevantes en la actualidad dentro del ámbito de la informática y la ingeniería del software. Según la Comisión Europea, su objetivo principal consiste en desarrollar sistemas capaces de realizar tareas que tradicionalmente requieren inteligencia humana, como el razonamiento, el aprendizaje, la percepción o la toma de decisiones (Gobierno de España, 2023). A lo largo de las últimas décadas, esta área ha experimentado una evolución marcada por avances técnicos, periodos de estancamiento y grandes hitos que han permitido consolidarla como una tecnología clave en múltiples sectores.

El origen formal del término Artificial Intelligence se sitúa en el año 1956, durante el Dartmouth Summer Research Project on Artificial Intelligence (McCarthy, Minsky, Rochester, & Shannon, 2006), un taller de investigación organizado por John McCarthy, Marvin Minsky, Nathaniel Rochester y Claude Shannon. Este evento es ampliamente considerado como el nacimiento oficial de la IA como campo académico independiente, ya que estableció por primera vez la idea de que las máquinas podrían simular aspectos del pensamiento humano mediante programación y modelos matemáticos.

En las décadas posteriores, la investigación en IA se centró principalmente en el desarrollo de sistemas simbólicos y basados en reglas, conocidos como *expert systems*, que intentaban reproducir el conocimiento humano mediante representaciones lógicas. Bruce G. Buchanan y Reid G. Smith publicaron en 1988 un artículo titulado “*Fundamentals of Expert Systems*” (Buchanan & Smith, 1988), en el que describen estos sistemas como programas diseñados para resolver problemas complejos mediante conocimiento especializado, almacenado habitualmente en forma de reglas “if-then”. Los autores destacan que la característica fundamental de los sistemas expertos no es únicamente su capacidad de inferencia, sino la existencia de una base de conocimiento explícita, separada del motor de razonamiento, lo que permitía explicar decisiones y replicar el comportamiento de un experto humano en dominios concretos como la medicina o la ingeniería. Sin embargo, también señalan que estos enfoques resultaban difíciles de mantener y escalar, ya que dependían de la adquisición manual de conocimiento y carecían de flexibilidad ante situaciones no previstas.

Durante los años 80 y 90, se produjo un cambio progresivo hacia enfoques basados en el aprendizaje automático, donde las máquinas comienzan a extraer patrones a partir de datos

en lugar de depender únicamente de reglas explícitas. Un hito representativo de este periodo fue la victoria del superordenador Deep Blue de IBM frente al campeón mundial de ajedrez Garry Kasparov en 1997 (Wikipedia, 2007). Este acontecimiento tuvo un importante impacto mediático y simbólico, ya que evidenció el potencial de las máquinas para superar capacidades humanas en tareas complejas.

Sin embargo, el verdadero impulso moderno de la IA se produjo a partir de la década de 2010 con el auge del aprendizaje profundo, basado en redes neuronales artificiales de múltiples capas. Este avance fue posible gracias al incremento de la potencia computacional, la disponibilidad de grandes volúmenes de datos y el desarrollo de nuevas arquitecturas más eficientes. Tal y como señalan LeCun, Bengio y Hinton en su trabajo titulado “Deep Learning” (LeCun, Bengio, & Hinton, 2015), el aprendizaje profundo ha permitido que los sistemas aprendan representaciones jerárquicas complejas directamente a partir de los datos, impulsando de forma decisiva el rendimiento en tareas como visión, voz y procesamiento del lenguaje.

Un punto de inflexión clave ocurrió en 2012, cuando el modelo AlexNet, desarrollado por Krizhevsky, Sutskever y Hinton, obtuvo resultados significativamente superiores en el concurso ImageNet de clasificación de imágenes. Este logro marcó el inicio de la revolución de las redes neuronales convolucionales en tareas visuales, consolidando la visión por computador como una de las áreas más importantes dentro de la IA actual. En comparación con los métodos anteriores, los autores destacan que su red profunda logró reducir de forma considerable el error top-5, obteniendo un resultado muy por encima del estado del arte existente en ese momento, lo que evidenció por primera vez el potencial del aprendizaje profundo en reconocimiento visual a gran escala (Krizhevsky, Sutskever, & Hinton, 2012).

Posteriormente, en 2017, la publicación del modelo Transformer bajo el título “Attention Is All You Need” (Ashish, et al., 2017) supuso otro avance fundamental, introduciendo mecanismos de atención como base de arquitecturas modernas utilizadas tanto en procesamiento de lenguaje natural como en tareas multimodales. En comparación con los enfoques previos basados en redes recurrentes o convolucionales, se demostró que era posible prescindir completamente de la recurrencia y basar el modelo únicamente en mecanismos de autoatención, logrando un entrenamiento más eficiente y mejores resultados en tareas de traducción automática. Este cambio arquitectónico permitió un paralelismo

mucho mayor durante el entrenamiento y sentó las bases de los modelos de IA generalistas actuales.

En la actualidad, la Inteligencia Artificial se aplica de manera extensiva en campos como la medicina, la industria, la automoción y la ingeniería. La evolución histórica descrita demuestra cómo la IA ha pasado de ser una disciplina principalmente teórica a convertirse en una herramienta esencial para resolver problemas reales en múltiples sectores. En el caso de España, esta adopción también es creciente: según el informe del Observatorio Nacional de Tecnología y Sociedad (ONTSI, 2023), el 11,8% de las empresas españolas con más de diez trabajadores ya utilizan IA, destacando especialmente su aplicación en tareas de automatización y, en un 39,7% de los casos, en la identificación de personas u objetos mediante imágenes. Este último aspecto resulta especialmente relevante para el presente proyecto, ya que conecta directamente con la visión por computador, ámbito que constituye la base tecnológica del sistema de vigilancia doméstica propuesto.

2.2.2. Visión por computador

Según la Microsoft, la visión por computador es una rama de la Inteligencia Artificial cuyo objetivo es permitir que los sistemas informáticos interpreten y comprendan información visual procedente de imágenes o vídeos (Microsoft, s.f.). Mediante el uso de algoritmos matemáticos y modelos de aprendizaje automático, estos sistemas son capaces de extraer características relevantes, identificar objetos y reconocer patrones presentes en una escena.

En el ámbito de la videovigilancia, la visión por computador se utiliza para tareas como la detección de movimiento, el seguimiento de objetos y la identificación de personas. A diferencia de los enfoques tradicionales basados únicamente en diferencias entre fotogramas, las técnicas modernas permiten analizar el contenido semántico de la imagen, proporcionando resultados más robustos frente a cambios de iluminación o ruido visual. En este sentido, Advantage Tech (Advantage Technology, 2025) ha realizado una comparativa entre los sistemas de videovigilancia tradicionales y aquellos impulsados por Inteligencia Artificial, destacando que uno de los beneficios más relevantes es la reducción significativa de la necesidad de supervisión humana constante. Mientras que los sistemas convencionales requieren validación manual frecuente debido al elevado número de falsas alarmas, la videovigilancia basada en IA permite automatizar gran parte del proceso de detección e

interpretación de eventos, mejorando así la eficiencia operativa y la capacidad de respuesta ante posibles amenazas

2.2.3. Aprendizaje automático y redes neuronales profundas

La detección de personas es una tarea fundamental dentro de la visión por computador, cuyo objetivo consiste en identificar automáticamente la presencia de individuos en una imagen o en una secuencia de vídeo mediante la localización de regiones de interés, normalmente representadas como cuadros delimitadores (bounding boxes) y poder diferenciar entre estímulos irrelevantes del entorno y eventos potencialmente significativos relacionados con actividad humana.

Históricamente, la detección de personas se abordó mediante técnicas clásicas basadas en extracción manual de características. Métodos como Histogram of Oriented Gradients (HOG), combinados con clasificadores tradicionales como máquinas de vectores soporte, demostraron buenos resultados en escenarios controlados (Dalal & Triggs, 2005). Sin embargo, estos enfoques presentan limitaciones importantes en entornos reales, donde existen variaciones de escala, posturas complejas, oclusiones parciales y cambios en las condiciones de iluminación.

Con el avance del aprendizaje profundo, las redes neuronales convolucionales han pasado a dominar este campo gracias a su capacidad para aprender representaciones visuales jerárquicas directamente a partir de los datos. Uno de los primeros avances relevantes fue el desarrollo de modelos basados en regiones como R-CNN, propuesto por Girshick et al. (Girshick, Donahue, & Malik, 2014). Este enfoque supuso un punto de inflexión en la detección de objetos, ya que los autores demostraron que era posible combinar propuestas de regiones generadas de manera ascendente con redes convolucionales de alta capacidad, mejorando el rendimiento en el conjunto de referencia PASCAL VOC. En concreto, R-CNN consiguió aumentar el mean average precision en más de un 30% respecto al mejor resultado previo, alcanzando un mAP de 53.3% en VOC 2012. Además, el trabajo introdujo la idea de aprovechar el preentrenamiento supervisado y el ajuste fino posterior como estrategia clave cuando los datos etiquetados disponibles son limitados, sentando las bases de los detectores modernos basados en aprendizaje profundo.

No obstante, en sistemas de videovigilancia doméstica resulta especialmente importante disponer de modelos capaces de operar en tiempo real y con recursos computacionales limitados. En este contexto destacan los detectores de una sola etapa (single-stage detectors), como SSD (Single Shot Detector), que permiten realizar detección directamente en una única pasada por la red, reduciendo significativamente la latencia frente a modelos de dos etapas. Liu et al. (Liu W. , y otros, 2016) proponen SSD como un enfoque basado en una única red neuronal profunda que elimina completamente la necesidad de generar propuestas de regiones, encapsulando todo el proceso de detección en un único modelo. El método discretiza el espacio de salida mediante un conjunto de default bounding boxes con diferentes escalas y proporciones, combinando predicciones de múltiples mapas de características para gestionar objetos de distintos tamaños. Los autores demuestran que SSD mantiene una precisión competitiva frente a detectores basados en propuestas, alcanzando un mAP de 74.3% en VOC2007 a 59 FPS, y superando incluso a Faster R-CNN en determinadas configuraciones, lo que lo convierte en una solución especialmente adecuada para aplicaciones en tiempo real.

A partir de esta arquitectura, se han desarrollado variantes optimizadas para dispositivos embebidos, como MobileNet-SSD, que combina el enfoque SSD con redes ligeras basadas en convoluciones separables en profundidad. Estas redes reducen drásticamente el número de parámetros y operaciones necesarias, manteniendo un rendimiento adecuado para tareas prácticas en hardware de bajos recursos (Howard, y otros, 2017).

En el marco del presente proyecto, la detección de personas constituye el primer módulo inteligente del sistema de vigilancia propuesto. Su función es actuar como filtro semántico inicial, activando los procesos posteriores únicamente cuando se confirma la presencia de un individuo en la escena. Esta estrategia permite reducir falsas alarmas y mejorar la eficiencia global del sistema, ya que evita procesamientos innecesarios en ausencia de actividad humana.

De este modo, la detección basada en MobileNet-SSD se presenta como una solución adecuada para sistemas de videovigilancia locales, donde es imprescindible equilibrar precisión, velocidad de inferencia y viabilidad computacional.

2.2.4. Protocolo RTSP

2.2.4.1. Origen y propiedades

El RTSP (Real Time Streaming Protocol) es un protocolo de control de nivel aplicación diseñado para la transmisión de contenido multimedia en tiempo real a través de redes IP. Fue definido inicialmente por la Internet Engineering Task Force (IETF) y estandarizado en la RFC 2326 (Schulzrinne H. R., 1998), con actualizaciones posteriores en la RFC 7826 (Schulzrinne H. e., 2016). RTSP se utiliza ampliamente en sistemas de videovigilancia, streaming profesional, pruebas de visión artificial y aplicaciones multimedia en tiempo real.

A diferencia de protocolos orientados a la transferencia de archivos, como HTTP, RTSP no transporta directamente los datos multimedia, sino que actúa como un protocolo de control que gestiona sesiones de streaming. Los flujos de audio y vídeo suelen transmitirse mediante RTP (Real-time Transport Protocol) y RTCP (RTP Control Protocol), mientras que RTSP se encarga de establecer, controlar y finalizar dichas sesiones.

Sigue un modelo cliente-servidor, en el cual el cliente (por ejemplo, un reproductor multimedia o una aplicación de análisis de vídeo) envía comandos al servidor RTSP para controlar la reproducción del contenido. La Tabla 1 muestra los comandos disponibles del protocolo, donde C es el cliente, S es el servidor, P es presentación (recurso completo) y S es stream (flujo individual). (Schulzrinne H. R., 1998).

<i>Comando</i>	<i>Dirección</i>	<i>Objeto</i>	<i>Descripción</i>
<i>DESCRIBE</i>	C->S	P, S	Solicita la descripción del flujo multimedia (generalmente en formato SDP).
<i>ANNOUNCE</i>	C->S, S->C	P, S	Envía al receptor la descripción de una sesión o flujo multimedia; se utiliza principalmente cuando el cliente desea publicar contenido en el servidor.
<i>GET_PARAMETERS</i>	C->S, S->C	P, S	Solicita o devuelve parámetros asociados a la sesión o al flujo, como estadísticas o información específica del servidor.

<i>OPTIONS</i>	C->S, S->C	P, S	Consulta los métodos RTSP soportados por el servidor o cliente.
<i>SETUP</i>	C->S	S	Negocia los parámetros de transporte (puertos y protocolos).
<i>PLAY</i>	C->S	P, S	Inicia o reanuda la transmisión.
<i>PAUSE</i>	C->S	P, S	Detiene temporalmente la reproducción.
<i>RECORD</i>	C->S	P, S	Solicita al servidor que comience a recibir y almacenar el flujo multimedia enviado por el cliente.
<i>REDIRECT</i>	S->C	P, S	Informa al cliente que el contenido solicitado se ha movido a una nueva ubicación RTSP.
<i>SET_PARAMETER</i>	C->S, S->C	P, S	Establece o actualiza parámetros asociados a la sesión, como opciones de control o valores definidos por la aplicación.
<i>TEARDOWN</i>	C->S	P, S	Finaliza la sesión.

Tabla 1. Comandos del protocolo RTSP

Este enfoque permite un control similar al de un reproductor multimedia tradicional (play, pause, stop), pero aplicado a flujos en tiempo real sobre red.

2.2.4.2. Diferencias con otros protocolos

RTSP se diferencia de otros protocolos de transmisión como HTTP Live Streaming (HLS) o MPEG-DASH en varios aspectos clave (Wonza Media Systems, 2025):

- **Baja latencia:** RTSP está diseñado para aplicaciones en tiempo real, mientras que HLS y DASH priorizan la escalabilidad y tolerancia a fallos.
- **Control interactivo:** RTSP permite controlar el flujo en tiempo real (pausa, avance, retroceso).

- **Uso en sistemas embebidos:** es ampliamente utilizado en cámaras IP, sistemas industriales y entornos de laboratorio.

2.2.4.3. Por qué es útil para el proyecto

El uso del protocolo RTSP resulta especialmente adecuado para el desarrollo de este proyecto, ya que permite desacoplar el sistema de vigilancia del dispositivo de captura de vídeo. Mediante RTSP, las cámaras IP se encargan de la codificación y transmisión del flujo de vídeo, reduciendo la carga de procesamiento en el sistema que ejecuta el análisis inteligente.

Este enfoque es especialmente relevante en entornos con recursos computacionales limitados, ya que el sistema receptor no necesita gestionar tareas intensivas como la captura directa de vídeo o la codificación de la señal. En su lugar, recibe un flujo ya procesado, que puede ser analizado de forma eficiente por los módulos de detección y reconocimiento.

Además, RTSP es un estándar ampliamente adoptado por cámaras domésticas y profesionales, lo que garantiza compatibilidad con una gran variedad de dispositivos y facilita la escalabilidad del sistema a múltiples cámaras sin requerir hardware específico. Esta característica refuerza la viabilidad del sistema en escenarios domésticos reales, donde el uso de equipos de bajo consumo y bajo coste es un requisito fundamental.

2.2.5. Procesamiento local y eficiencia computacional

La ejecución de modelos de visión por computador en entornos locales requiere un enfoque orientado a la eficiencia computacional, especialmente cuando se pretende desplegar el sistema en hardware de bajos recursos. A diferencia de infraestructuras en la nube, los dispositivos domésticos presentan limitaciones importantes en capacidad de procesamiento, memoria y consumo energético, lo que condiciona la elección de modelos y técnicas de optimización.

En este contexto, el paradigma conocido como Edge AI plantea la inferencia directamente en el dispositivo, reduciendo latencias y evitando dependencias externas. Shi et al. (Weisong, Jie, Quan, Youhuizi, & Lanyu, 2016) destacan que el edge computing permite ejecutar aplicaciones inteligentes en tiempo real mediante arquitecturas distribuidas cercanas a la fuente de datos.

Para hacer viable este tipo de procesamiento, se han desarrollado redes neuronales ligeras como MobileNet, diseñadas específicamente para minimizar el número de operaciones

mediante convoluciones separables en profundidad (Howard, y otros, 2017). Asimismo, técnicas como la cuantización o la poda de parámetros permiten reducir el tamaño de los modelos manteniendo un rendimiento adecuado (Song, Huizi, & Dally, 2015).

2.3.Aplicaciones relacionadas

En este apartado se analizan las principales soluciones existentes en el ámbito de la vigilancia doméstica con el objetivo de contextualizar el sistema propuesto y justificar la aportación del presente proyecto. Las alternativas disponibles pueden agruparse, de forma general, en dos grandes categorías: sistemas ofrecidos por empresas mediante suscripción y soluciones privadas que no requieren cuotas periódicas.

2.3.1. Sistemas de empresas con suscripción

La opción más extendida entre los usuarios es la contratación de servicios de vigilancia proporcionados por empresas especializadas. Estas soluciones suelen ofrecer un paquete completo, que incluye la instalación de cámaras, sensores de movimiento, sistemas de alarma y una central receptora que gestiona las alertas. Todo el proceso, desde la instalación inicial hasta el mantenimiento y la monitorización continua, queda en manos de la empresa proveedora. Su uso no hace más que aumentar con el tiempo, según Escudo Digital, el mercado de sistemas de seguridad en España alcanzó en 2024 un volumen de negocio de 3090 millones de euros, lo que supone un 7% respecto al ejercicio anterior. (Montes, 2025)

Este tipo de sistemas presenta ventajas claras. El usuario no necesita conocimientos técnicos ni preocuparse por la configuración del sistema, ya que la empresa se encarga de todos los aspectos relacionados con su funcionamiento. Además, suelen ofrecer soporte técnico continuo y respuesta ante incidentes, lo que proporciona una sensación de seguridad inmediata.

No obstante, estas soluciones también presentan importantes desventajas. En primer lugar, requieren el pago de cuotas periódicas, lo que supone un coste recurrente a largo plazo. Aproximadamente, para una casa media en España, puede suponer en torno a 50 euros al mes. (Loza, s.f.). En segundo lugar, y más relevante desde el punto de vista de este proyecto, implican una pérdida casi total de control sobre los datos. Las imágenes y vídeos capturados por las cámaras son procesados y, en muchos casos, almacenados en infraestructuras

externas, lo que plantea serias dudas en términos de privacidad y protección de la información personal.

La dependencia de servicios en la nube y de plataformas propietarias limita además la capacidad del usuario para personalizar el comportamiento del sistema o adaptar su funcionamiento a necesidades específicas. Estas limitaciones constituyen una de las principales motivaciones para el desarrollo de la solución propuesta en este proyecto.

2.3.2. Soluciones privadas, pero sin suscripción

Además de los sistemas basados en suscripción, existen soluciones comerciales que permiten al usuario adquirir directamente el hardware de vigilancia, como cámaras IP, sin necesidad de contratar una cuota mensual obligatoria. Empresas de este tipo ofrecen dispositivos que pueden instalarse de forma sencilla en el entorno doméstico y que permiten al usuario acceder al vídeo en tiempo real desde cualquier ubicación a través de aplicaciones móviles o interfaces web.

Estas soluciones suelen apoyarse en infraestructuras en la nube proporcionadas por el propio fabricante, que actúan como intermediarias para el acceso remoto a las cámaras. De este modo, el usuario puede visualizar las imágenes y gestionar el sistema sin necesidad de realizar configuraciones complejas de red, como redirección de puertos o servicios adicionales.

En muchos casos, el procesamiento de las imágenes y del vídeo se realiza en los servidores del fabricante, donde se aplican algoritmos de detección de movimiento, identificación de personas u otras funciones de análisis inteligente. Este enfoque permite ofrecer funcionalidades avanzadas sin requerir hardware potente en el domicilio del usuario.

No obstante, este modelo también presenta limitaciones relevantes. Aunque no exista una suscripción obligatoria para el uso básico del dispositivo, el usuario pierde el control directo sobre el tratamiento de los datos, ya que las imágenes capturadas deben ser enviadas a servidores externos para su análisis. Esta dependencia de la nube plantea preocupaciones en términos de privacidad, disponibilidad del servicio y dependencia del proveedor.

Adicionalmente, muchos de estos sistemas ofrecen planes de pago opcionales para el almacenamiento de grabaciones en la nube, el acceso a historiales ampliados o funcionalidades avanzadas de análisis. Esto introduce costes recurrentes si se desea un uso completo del sistema y refuerza la dependencia de servicios propietarios.

En comparación con estas soluciones, el sistema desarrollado en este trabajo propone un enfoque alternativo basado en procesamiento completamente local, evitando el envío de datos a infraestructuras externas. La detección de personas y el reconocimiento facial se ejecutan en el propio dispositivo, lo que garantiza un mayor control sobre la información, elimina la dependencia de servidores de terceros y permite adaptar el comportamiento del sistema a las necesidades específicas del usuario.

2.3.3. Blue Iris Software

Blue Iris es una de las soluciones comerciales más conocidas dentro del ámbito de la videovigilancia doméstica y profesional, especialmente orientada a usuarios que buscan un sistema centralizado de grabación y monitorización mediante cámaras IP. Se trata de un software que permite gestionar múltiples fuentes de vídeo, almacenar grabaciones localmente y acceder a las cámaras de forma remota a través de una interfaz unificada (Blueiris, 2026).

Entre sus principales ventajas destaca su alto nivel de compatibilidad con cámaras ONVIF y RTSP, así como la posibilidad de configurar alertas, grabación continua o por eventos y almacenamiento en servidores locales. Además, en versiones recientes incorpora funcionalidades basadas en Inteligencia Artificial para mejorar la detección de personas u objetos mediante integración con modelos externos.

Desde el punto de vista de la privacidad, Blue Iris ofrece una ventaja respecto a otras plataformas comerciales basadas en la nube, ya que permite mantener las grabaciones y el procesamiento en servidores locales. No obstante, su acceso remoto suele requerir configuraciones adicionales o servicios externos, lo que puede introducir riesgos si no se gestiona adecuadamente en términos de ciberseguridad. En cuanto al coste, se trata de una solución de pago bajo licencia, lo que puede suponer una barrera para usuarios que buscan alternativas abiertas o de bajo coste. Además, su ejecución eficiente suele necesitar hardware relativamente potente, especialmente cuando se manejan múltiples cámaras y funciones avanzadas simultáneamente.

2.4.PROYECTOS RELACIONADOS

2.4.1. Video Surveillance System Using IP Camera for Target Person Detection

Un proyecto relacionado con el presente trabajo es el sistema propuesto por Yimyam y Ketcham (Yimyam & Ketcham, 2018) orientado al desarrollo de una plataforma de videovigilancia basada en cámaras IP para la detección de una persona objetivo. El objetivo principal de dicha propuesta es identificar automáticamente a un individuo concreto dentro de un entorno supervisado, combinando reconocimiento facial con herramientas adicionales de localización geográfica enfocado a un uso policial para identificar sospechosos. Los autores emplean cámaras conectadas mediante el protocolo RTSP para capturar vídeo en tiempo real, aplicando técnicas de procesamiento de imagen con OpenCV. La identificación se realiza a través de clasificadores en cascada tipo Haar, una aproximación clásica ampliamente utilizada en detección facial. El sistema se complementa con un módulo de geolocalización que permite mostrar la posición del individuo detectado en un mapa.

Respecto a los resultados, el trabajo demuestra la viabilidad de integrar cámaras IP con reconocimiento facial en un entorno funcional, logrando detectar rostros y asociarlos a una persona objetivo. No obstante, los autores se apoyan en técnicas tradicionales y no profundizan en el uso de modelos neuronales optimizados o en arquitecturas orientadas a la eficiencia computacional.

El sistema desarrollado en este proyecto comparte la idea de automatizar la vigilancia mediante análisis visual, pero se enfoca específicamente en el ámbito doméstico, priorizando el procesamiento completamente local y el uso de modelos modernos de detección más adecuados para funcionar en dispositivos de bajos recursos. Además, se incorpora un dashboard web de administración que facilita al usuario final la gestión del sistema, la visualización de eventos y la configuración de alertas de manera accesible.

2.5.CONCLUSIONES

En este capítulo se ha presentado el contexto y estado del arte que fundamenta el desarrollo del sistema de vigilancia doméstica propuesto. A través de la revisión histórica de la Inteligencia Artificial y su evolución hacia la visión por computador, se han introducido los conceptos principales que permiten aplicar técnicas modernas de detección de personas en sistemas reales.

Asimismo, se han analizado tecnologías clave como el protocolo RTSP y las estrategias de procesamiento local mediante modelos eficientes, destacando su importancia para lograr una solución autónoma ejecutable en dispositivos de recursos limitados. Finalmente, el estudio de aplicaciones comerciales y proyectos relacionados ha permitido identificar limitaciones habituales en coste, dependencia externa y control de datos, justificando la necesidad de un enfoque alternativo orientado al ámbito doméstico y privado. Este capítulo establece la base teórica y comparativa sobre la que se construye el desarrollo software presentado en los capítulos posteriores.

3. OBJETIVOS CONCRETOS Y METODOLOGÍA DE TRABAJO

3.1.OBJETIVO GENERAL

El objetivo general de este proyecto es mejorar la seguridad sin renuncias a la privacidad en entornos domésticos mediante el desarrollo de un sistema de videovigilancia inteligente que funcione de forma completamente local, capaz de detectar la presencia de personas y generar alertas automáticas con un nivel reducido de falsas alarmas, ejecutándose en dispositivos de bajos recursos computacionales.

Para ello, se pretende demostrar que es posible aplicar modelos de visión por computador optimizados en un entorno privado sin dependencia de la nube, ofreciendo además una interfaz web de administración que permita al usuario supervisar y gestionar el sistema de manera sencilla. La efectividad de la solución se evaluará a través de pruebas de funcionamiento en tiempo real, comprobando su capacidad de detección y su viabilidad de ejecución en hardware doméstico accesible.

3.2.OBJETIVOS ESPECÍFICOS

Con el fin de alcanzar el objetivo general planteado, este proyecto se divide en un conjunto de objetivos específicos que permiten abordar el desarrollo de manera estructurada y evaluable. Estos objetivos representan las principales tareas necesarias para diseñar, implementar y validar el sistema de vigilancia propuesto.

Los objetivos específicos del trabajo son los siguientes:

- Analizar el estado del arte de los sistemas actuales de videovigilancia doméstica y de las técnicas de visión por computador aplicadas a la detección de personas.
- Identificar modelos de aprendizaje profundo eficientes que permitan ejecutar tareas de detección en tiempo real sobre hardware de recursos limitados.
- Desarrollar un módulo funcional de detección de personas en secuencias de vídeo procedentes de cámaras IP mediante protocolos de streaming.
- Integrar los distintos componentes en una arquitectura software modular con una interfaz web de administración para la supervisión del sistema y la gestión de eventos.
- Evaluar el rendimiento del sistema en un entorno doméstico, midiendo su eficacia en la reducción de falsas alarmas y su viabilidad de ejecución local en tiempo real.

Además del desarrollo técnico, el proyecto persigue documentar de forma detallada todas las decisiones tomadas, justificando la elección de tecnologías, modelos y métricas empleadas. De este modo, el trabajo no solo aporta una solución funcional, sino también un recurso académico que puede servir como base para desarrollos futuros o ampliaciones del sistema.

3.3. Metodología del trabajo

Para alcanzar los objetivos específicos definidos en este trabajo y, con ello, cumplir el objetivo general del proyecto, se ha seguido una metodología de desarrollo de tipo ágil basada en el marco de trabajo Scrum. Este enfoque permite avanzar de forma incremental, validar resultados de manera continua e introducir mejoras progresivas durante el desarrollo.

Scrum se basa en ciclos iterativos denominados sprints, en los que se implementan funcionalidades concretas y se evalúan de forma periódica. Esta metodología ha sido seleccionada debido a la naturaleza multidisciplinar del proyecto, que combina visión por computador, aprendizaje automático y desarrollo web, requiriendo integración continua y pruebas frecuentes en un entorno real.

La metodología se ha estructurado en varias fases principales, alineadas con los distintos sprints del desarrollo, desde el análisis inicial hasta la evaluación final del sistema.

3.3.1. Fase 1: Análisis del problema y definición de requisitos

El primer paso consistió en analizar el problema de partida y definir el alcance del sistema de videovigilancia. Esta fase establece qué funcionalidades debe cubrir la solución y qué restricciones condicionan su desarrollo, especialmente en términos de ejecución local y limitaciones computacionales.

Como resultado, se identificaron los requisitos funcionales y no funcionales del sistema, así como los escenarios domésticos que posteriormente se emplearían para evaluar su comportamiento. Esta información permitió construir un primer product backlog con las tareas a desarrollar en los sprints posteriores.

3.3.2. Fase 2: Investigación y selección de modelos de IA

Una vez delimitado el alcance, se llevó a cabo un análisis comparativo de técnicas de visión por computador aplicables a la detección de personas. El objetivo de esta fase fue seleccionar

modelos que ofrecieran un equilibrio adecuado entre precisión y eficiencia, requisito indispensable para funcionar en hardware de bajos recursos.

Para ello, se emplearon instrumentos como documentación científica, modelos preentrenados disponibles y librerías como OpenCV y frameworks de inferencia optimizados.

La decisión de utilizar transferencia de aprendizaje permitió evitar entrenamientos costosos y facilitar la integración de modelos robustos en un entorno doméstico.

3.3.3. Fase 3: Diseño incremental de la arquitectura del sistema

Siguiendo Scrum, el diseño de la arquitectura se planteó de forma modular e incremental, estableciendo componentes independientes que pudieran desarrollarse y probarse en sprints sucesivos.

En esta fase se definieron los principales módulos del sistema: captura de vídeo RTSP, detección de personas, gestión de eventos, notificaciones y dashboard web. Asimismo, se establecieron las estructuras necesarias para el almacenamiento local y la configuración del sistema.

3.3.4. Fase 4: Implementación iterativa mediante sprints

El desarrollo del software se realizó a través de varios sprints, implementando en cada ciclo una parte funcional del sistema. Esta estrategia permitió validar cada componente antes de integrarlo en el flujo completo.

Los instrumentos utilizados incluyeron Python como lenguaje principal, FastAPI para el desarrollo web, Docker para el despliegue y librerías de visión artificial para el procesamiento de vídeo. Al final de cada sprint se realizaban pruebas funcionales para verificar que los módulos desarrollados cumplieran su propósito.

3.3.5. Fase 5: Evaluación experimental y análisis de métricas

Con el sistema completo, se diseñaron pruebas en escenarios domésticos representativos, incluyendo presencia de propietarios, intrusos simulados, mascotas y variaciones de iluminación.

El objetivo de esta fase fue analizar el rendimiento del sistema mediante métricas observables, como:

- tasa de detección de personas
- número de falsas alarmas
- veces que el sistema no detecto un individuo que sí existía

Los resultados obtenidos se recopilaron y compararon con los requisitos definidos inicialmente, permitiendo valorar objetivamente la eficacia del sistema desarrollado.

3.3.6. Fase 6: Análisis final y documentación del proyecto

Finalmente, se interpretaron los resultados obtenidos durante la evaluación para determinar el grado de cumplimiento de los objetivos del trabajo. Esta fase incluye la identificación de limitaciones actuales y posibles mejoras futuras.

4. IDENTIFICACIÓN DE REQUISITOS

Antes de iniciar el desarrollo de la herramienta software, es necesario identificar los requisitos que debe cumplir el sistema para responder al problema planteado. En este proyecto, el objetivo es disponer de un sistema de videovigilancia doméstica inteligente que funcione de manera local, capaz de detectar la presencia de personas y generar alertas relevantes, reduciendo falsas alarmas y evitando la dependencia de servicios externos.

A partir del análisis del contexto de uso y de soluciones existentes, se definieron los requisitos funcionales y no funcionales que han guiado el diseño e implementación del sistema.

4.1. REQUISITOS FUNCIONALES

Los requisitos funcionales describen las capacidades principales que debe ofrecer el sistema desarrollado:

- **RF1.** Captura de vídeo en tiempo real: el sistema debe ser capaz de recibir secuencias de vídeo procedentes de cámaras IP mediante el protocolo RTSP.
- **RF2.** Detección automática de personas: el sistema debe identificar la presencia de personas en los fotogramas analizados utilizando modelos de visión por computador.
- **RF3.** Generación de eventos de vigilancia: cuando se detecte una persona, el sistema debe registrar el evento correspondiente para su posterior consulta.
- **RF4.** Envío de alertas al usuario: el sistema debe notificar automáticamente al usuario ante eventos relevantes mediante un canal externo, como Telegram.
- **RF5.** Interfaz web de administración: el sistema debe incluir un dashboard accesible desde el navegador que permita supervisar la actividad y consultar eventos registrados.
- **RF6.** Configuración básica del sistema: el usuario debe poder gestionar parámetros esenciales como cámaras conectadas o activación de notificaciones.

4.2. REQUISITOS NO FUNCIONALES

Los requisitos no funcionales establecen las restricciones y propiedades de calidad que debe cumplir el sistema:

- **RNF1.** Procesamiento local: todo el análisis de vídeo debe realizarse en el entorno local, sin enviar imágenes ni grabaciones a servicios en la nube.
- **RNF2.** Compatibilidad con hardware limitado: el sistema debe ser ejecutable en dispositivos domésticos de bajos recursos, utilizando modelos ligeros y optimizados.
- **RNF3.** Tiempo de respuesta adecuado: la detección y generación de alertas debe realizarse con baja latencia para permitir una reacción rápida ante intrusiones.
- **RNF4.** Reducción de falsas alarmas: el sistema debe minimizar notificaciones innecesarias mediante detección semántica de personas frente a simple movimiento.
- **RNF5.** Modularidad y extensibilidad: la arquitectura software debe permitir incorporar mejoras futuras, como nuevos modelos o funcionalidades adicionales.
- **RNF6.** Usabilidad: la interfaz web debe ser sencilla e intuitiva para facilitar el uso por parte de usuarios no expertos.

5. DESCRIPCIÓN DE LA HERRAMIENTA DE SOFTWARE DESARROLLADA

5.1. ENTORNO DE DESARROLLO

La idea final es que la aplicación pueda ser ejecutada en servidores (ordenadores) basados en Linux de bajos recursos, como puede ser una Raspberry Pi o un mini ordenador. Para facilitar el desarrollo, se ha decidido que se hará en un ordenador convencional y las pruebas se realizarán en una máquina virtual simulando un servidor casero.

Es importante remarcar que las pruebas finales si se realizarán en un entorno lo más posible al uso real que se espera dar a la aplicación, con cámaras reales y un servidor local.

5.1.1. Ordenador de desarrollo

El ordenador principal que se ha utilizado para el desarrollo es un Lenovo Thinkpad E16 Gen 2 con las características indicadas en la Tabla 2:

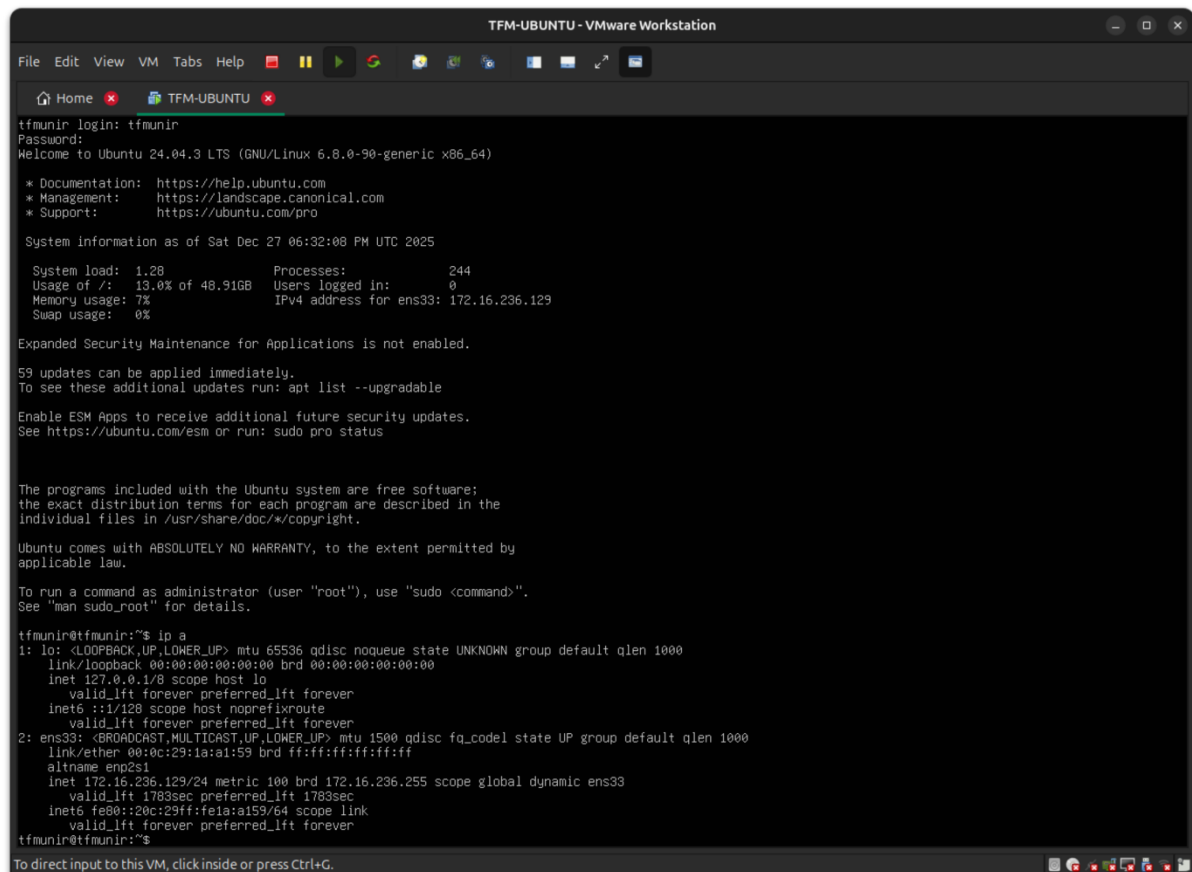
Tabla 2. Características del ordenador de desarrollo

Modelo	Lenovo Thinkpad E16 Gen 2
Sistema Operativo	Ubuntu 24.04.3 LTS
Procesador	AMD Ryzen 7 7735HS
RAM	32GB

El equipo utilizado durante el desarrollo dispone de recursos significativamente superiores a los del servidor objetivo donde se desplegará finalmente el sistema. A modo de ejemplo, una Raspberry Pi 5 cuenta con 4 GB de memoria RAM, frente a los 32 GB disponibles en la máquina de desarrollo. Esta diferencia resulta especialmente relevante durante el proceso de implementación, ya que es necesario verificar con frecuencia que la aplicación funcione correctamente en un entorno más modesto y representativo del sistema final. Con este propósito, se ha configurado una máquina virtual con recursos limitados, cuyas especificaciones se detallan en la siguiente sección.

5.1.2. Máquina virtual servidor (Ubuntu Server 24.04.3)

A continuación, se presenta las características elegidas al crear la máquina virtual que ha servido como servidor para las pruebas. La ventaja de usar una máquina virtual, principalmente, es que los recursos se pueden incrementar o reducir fácilmente para probar múltiples escenarios. La Ilustración 1 muestra la máquina virtual corriendo en VMWare.



```
TFM-UBUNTU - VMware Workstation
File Edit View VM Tabs Help
Home x TFM-UBUNTU x
tfmunir login: tfmunir
Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-90-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sat Dec 27 06:32:08 PM UTC 2025

System load:  1.28           Processes:    244
Usage of /:   13.0% of 48.91GB Users logged in: 0
Memory usage: 7%            IPv4 address for ens33: 172.16.236.129
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

59 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

tfmunir@tfmunir:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:1a:a1:59 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 172.16.236.129/24 metric 100 brd 172.16.236.255 scope global dynamic ens33
        valid_lft 1783sec preferred_lft 1783sec
    inet6 fe80::20c:29ff:fe1a:a159/64 scope link
        valid_lft forever preferred_lft forever
tfmunir@tfmunir:~$
```

Ilustración 1. Máquina virtual Ubuntu corriendo

La máquina virtual se configuró inicialmente con 8 GB de memoria RAM, dos procesadores y un núcleo por procesador, con el objetivo de simular un entorno de recursos limitados comparable al de una Raspberry Pi.

5.1.3. Raspberry Pi 5

Como sistema final de pruebas, y con el objetivo de simular de la forma más realista posible el entorno en el que un usuario doméstico podría desplegar la aplicación, se utilizó una Raspberry Pi 5 con 8 GB de memoria RAM. Este dispositivo representa una plataforma adecuada para evaluar la solución desarrollada, ya que combina un tamaño reducido, bajo consumo energético y un coste accesible.

Además, la Raspberry Pi 5 ofrece una mejora significativa en capacidad de procesamiento respecto a generaciones anteriores, lo que permite ejecutar modelos ligeros de visión por computador manteniendo tiempos de respuesta aceptables. Su utilización como entorno de pruebas resulta clave para comprobar la viabilidad del sistema en condiciones reales, verificando tanto el rendimiento de inferencia como la estabilidad del procesamiento continuo de vídeo en hardware limitado.

5.1.4. Cámaras de video

Para el desarrollo del proyecto se han empleado diferentes tipos de cámaras, seleccionadas en función de la fase del trabajo y del entorno de pruebas considerado. Esta combinación ha permitido validar el sistema tanto en un entorno de desarrollo controlado como en un escenario doméstico realista, asegurando que la solución propuesta no depende de un único tipo de dispositivo de captura.

Durante la fase inicial de diseño e implementación, las dos cámaras principales utilizadas fueron la webcam integrada del propio ordenador de desarrollo y una webcam externa Logitech C920. Ambas cámaras cuentan con conexión directa mediante interfaz USB y resultan especialmente adecuadas para tareas de prototipado y depuración, ya que ofrecen una integración sencilla con el sistema operativo y con las librerías de visión por computador utilizadas. Sin embargo, este tipo de dispositivos no emplea de forma nativa el protocolo RTSP descrito anteriormente, ya que están pensados para aplicaciones locales y no para la transmisión de vídeo en red. Esta limitación se solventa más adelante mediante la creación de un servidor RTSP intermedio, tal y como se detalla en el apartado 5.3, permitiendo que estas webcams puedan integrarse en el mismo flujo de trabajo que las cámaras de vigilancia reales.



Ilustración 2. Instalación de la cámara Tapo usada para pruebas en entorno real

Para las pruebas en un entorno más cercano al uso final del sistema se han utilizado dos cámaras de vigilancia doméstica de la marca Tapo, concretamente el modelo C510W (Ilustración 2). Estas cámaras están diseñadas específicamente para aplicaciones de seguridad y monitorización, e incorporan características habituales en este tipo de dispositivos, como conectividad inalámbrica, codificación de vídeo en tiempo real y funcionamiento autónomo. Desde el punto de vista de la clasificación realizada en el apartado 2.3.2, estas cámaras se encuadran dentro de la categoría de “soluciones privadas, pero sin suscripción”, ya que permiten su uso sin necesidad de contratar servicios externos obligatorios.

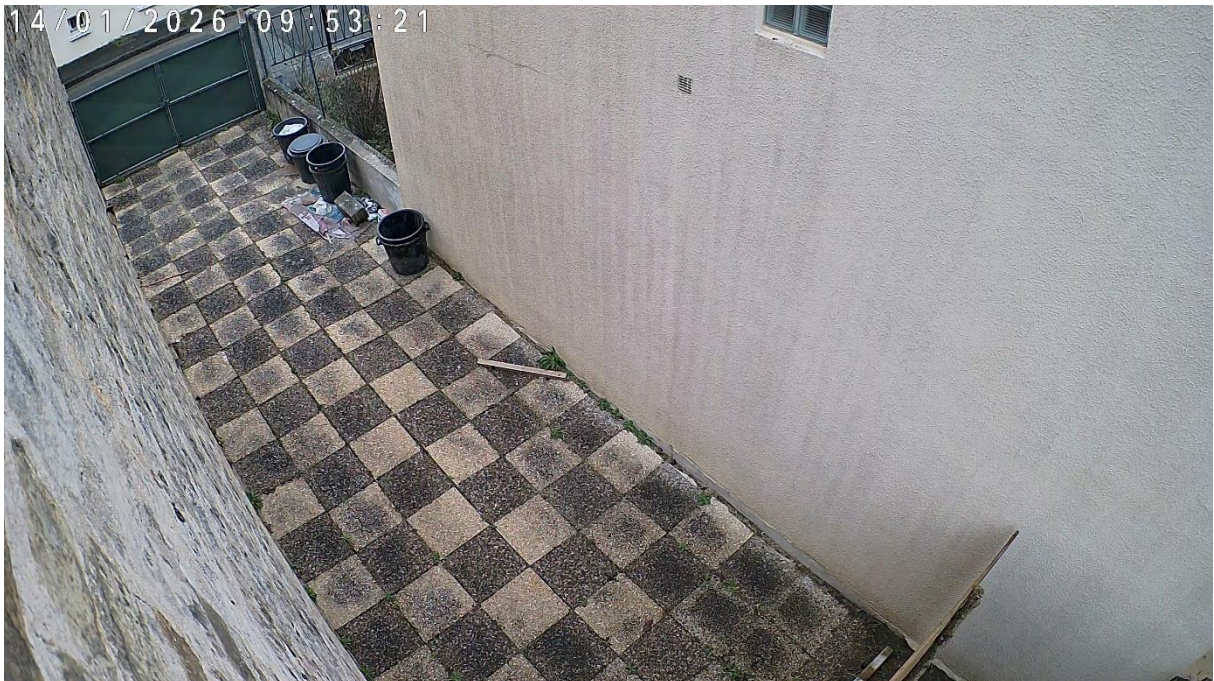


Ilustración 3. Captura de una cámara Tapo emitiendo video por RTSP

Una de las ventajas clave de este modelo concreto es la posibilidad de emitir el flujo de vídeo mediante el protocolo RTSP (Ilustración 3), lo que facilita su integración directa con el sistema desarrollado sin necesidad de componentes intermedios adicionales y sin depender de los servicios cloud de la empresa que las comercializa. Gracias a esta funcionalidad, las cámaras pueden actuar como fuentes RTSP nativas, proporcionando flujos de vídeo comprimidos que el sistema puede consumir directamente. Esta característica resulta especialmente relevante para validar el comportamiento del sistema en condiciones reales, donde la captura de vídeo se realiza de forma inalámbrica y con dispositivos típicos del ámbito doméstico.

La utilización combinada de webcams USB y cámaras de vigilancia RTSP ha permitido comprobar que la arquitectura propuesta es flexible y agnóstica respecto al dispositivo de captura, siempre que el flujo de vídeo pueda ser expuesto mediante RTSP.

5.2.ARQUITECTURA

En este apartado se presenta la arquitectura general del sistema desarrollado, representada de forma esquemática en la Ilustración 4. Se trata de una arquitectura de alto nivel cuyo objetivo es mostrar cómo se interconectan los distintos componentes del sistema y cómo se distribuyen las responsabilidades entre la red local y la red externa, manteniendo el procesamiento inteligente y los datos sensibles dentro del entorno doméstico.

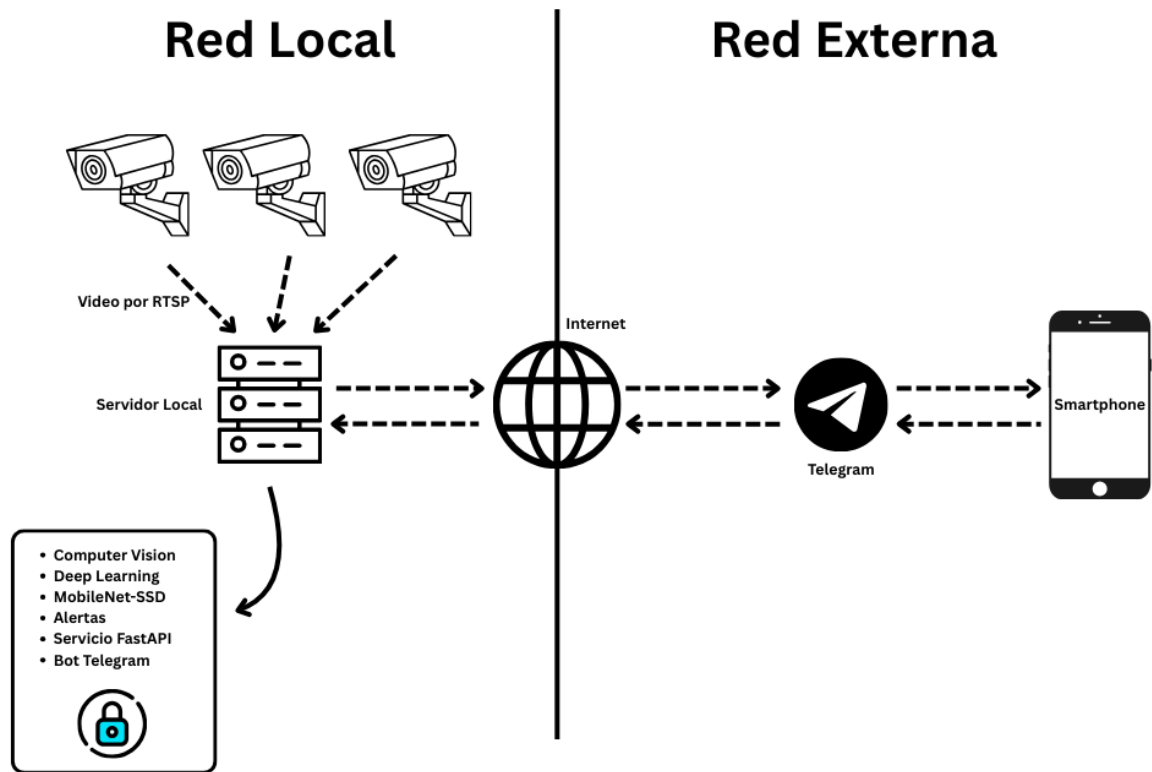


Ilustración 4. Esquema de la arquitectura a alto nivel de la solución

La arquitectura propuesta sigue un enfoque de procesamiento local, en el que las cámaras de vigilancia actúan como fuentes de vídeo y envían sus flujos al servidor local a través de la red doméstica, utilizando el protocolo RTSP. Este servidor constituye el núcleo del sistema y concentra las tareas de análisis inteligente del vídeo, evitando que las imágenes capturadas deban ser enviadas a servicios externos o infraestructuras en la nube.

Dentro del servidor local se articula un flujo de procesamiento secuencial que integra los distintos módulos funcionales del sistema. De forma general, este flujo abarca la captura del vídeo, su preprocesado, la detección de personas mediante modelos de aprendizaje profundo y la generación de eventos a partir de las detecciones confirmadas. Estos módulos se encuentran desacoplados y organizados de manera que facilitan la extensibilidad del sistema y la incorporación de nuevas funcionalidades en fases posteriores del desarrollo.

La gestión de alertas, el estado del sistema y la configuración de las cámaras se centraliza en un servicio backend desarrollado con FastAPI. Este servicio actúa como punto de coordinación entre los distintos módulos y proporciona una interfaz accesible desde la red local mediante

un dashboard web. A través de este panel, el usuario puede consultar las alertas generadas, revisar evidencias asociadas y gestionar los dispositivos conectados, ofreciendo una visión unificada del funcionamiento del sistema sin necesidad de interactuar directamente con los componentes internos.

El acceso desde la red externa se mantiene deliberadamente limitado y controlado. Tal y como se muestra en la Ilustración 4, el servidor local se comunica con el exterior únicamente a través de un bot de Telegram, que se utiliza como canal de notificación hacia el smartphone del usuario. Este mecanismo permite informar de la ocurrencia de eventos relevantes y aceptar comandos simples de consulta o control, sin exponer el sistema ni los flujos de vídeo al exterior.

Esta arquitectura proporciona una separación clara de responsabilidades entre captura, análisis, gestión y notificación, al tiempo que prioriza la privacidad y el control local de los datos. El diseño modular y a alto nivel facilita la comprensión del sistema en su conjunto y sienta la base para el desarrollo detallado de cada uno de los módulos descritos en los apartados siguientes.

5.3. CAPTURA DE VÍDEO

Para facilitar el desarrollo de la aplicación, se han usado cámaras web convencionales conectadas por USB a la máquina principal. En la sección 2.2.4 se ha hablado del protocolo RTSP, el cuál es vital para la aplicación ya que va a ser el protocolo utilizado para recibir el streaming de video de manera inalámbrica en el servidor, pero en este caso las cámaras están conectadas mediante USB.

5.3.1. Servidor RTSP MediaMTX

Para solventar este problema, se ha creado un servidor RTSP que toma el video de las cámaras USB y lo emite por una URL RTSP. Lo primero es instalar Mediamtx, un servidor multimedia ligero y de alto rendimiento que permite publicar, distribuir y convertir flujos de audio y vídeo en tiempo real. Actúa como punto central de streaming, aceptando flujos entrantes (ffmpeg o cámaras IP, por ejemplo) y redistribuyéndolos a múltiples clientes mediante distintos protocolos. (Bluenviron, 2026).

En particular, MediaMTX se usa ampliamente como servidor RTSP, donde gestiona la señalización RTSP y el transporte RTP/RTCP. Está diseñado para entornos de pruebas,

investigación y producción ligera, destacando su configuración sencilla, bajo consumo de recursos y compatibilidad multiplataforma (Olamide, 2025),

Para instalarlo:

```
wget  
https://github.com/bluenviron/mediamtx/releases/latest/download/mediamtx_linux_amd64.tar.gz  
tar -xzf mediamtx_linux_amd64.tar.gz  
sudo mv mediamtx /usr/local/bin/
```

Se descarga del repositorio oficial en GitHub (<https://github.com/bluenviron/mediamtx>) y se mueve a una localización que se encuentre en el PATH para que sea más fácil ejecutarlo desde cualquier sitio de la máquina.

MediaMTX sólo crea el servidor RTSP, ahora es necesario poder enviarle el flujo de video de las cámaras. Para ello se utiliza FFmpeg.

5.3.2. FFmpeg para manejar el flujo de vídeo

FFmpeg es un conjunto de herramientas multimedia de código abierto ampliamente utilizado para capturar, codificar, decodificar, transcodificar y transmitir audio y video. Proporciona una arquitectura modular basada en librerías, como libavcodec, libavformat y libavdevice, que permiten trabajar con una gran variedad de códecs, contenedores y protocolos de streaming (FFmpeg, 2025).

En el contexto de sistemas de streaming y experimentación, FFmpeg se emplea habitualmente como herramienta de captura y codificación, por ejemplo, para obtener vídeo desde webcams, cámaras IP o archivos multimedia, codificarlo en formatos como H.264/H.265, y publicarlo mediante protocolos como RTSP, RTP, RTMP o SRT. Su flexibilidad, rendimiento y portabilidad lo convierten en un componente fundamental en entornos de investigación, visión por computador y pruebas de transmisión en tiempo real.

Para instalarlo:

```
sudo apt update  
sudo apt install -y ffmpeg
```

5.3.3. Conectar FFmpeg y MediaMTX

Ahora que tanto MediaMTX y FFmpeg están instalados, se pueden conectar entre ellos para obtener una URL rtsp que emita el flujo de video proveniente de una cámara web conectada por USB.

Primero es necesario saber dónde se encuentra la cámara. En sistemas Linux, se pueden encontrar como un archivo en el path `/dev`. Si se observa la Ilustración 5, se puede ver las cámaras disponibles en mi computadora.

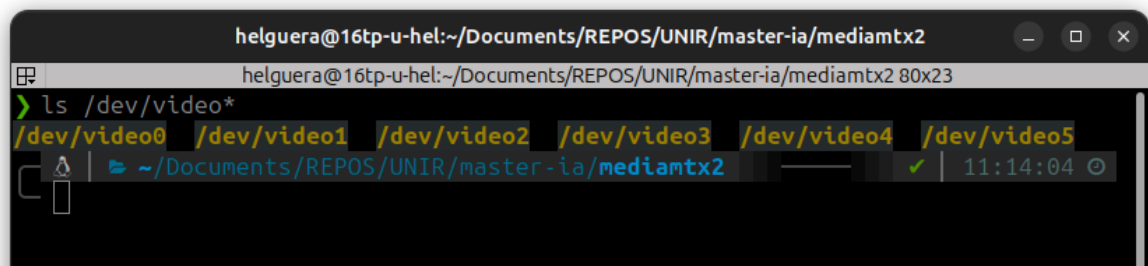


Ilustración 5. Lista de cámaras disponibles en `/dev/video*`

En mi caso concreto, video0 es la webcam integrada en la computadora y video4 es una webcam conectada por USB.

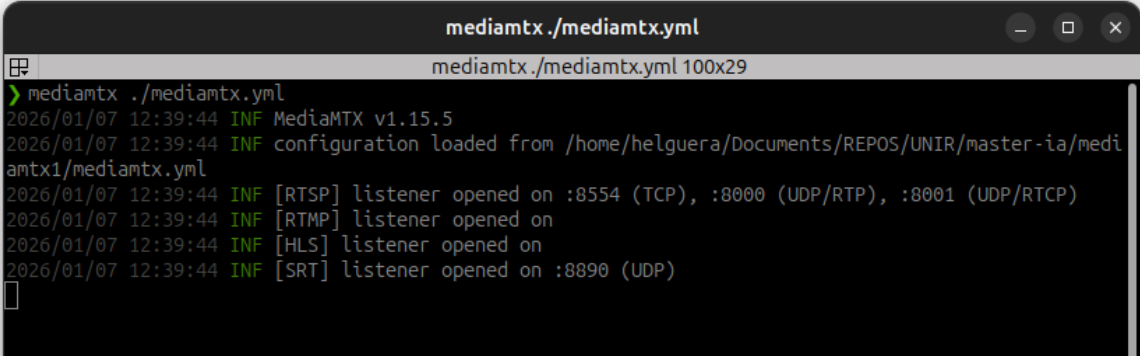
Para ejecutar el servidor MediaMTX, se requiere de un fichero `mediamtx.yml` con las configuraciones. Tiene que estar guardado en la misma ruta desde donde se ejecuta el comando de la terminal, no donde se encuentra el ejecutable. El fichero es el siguiente:

```
rtspAddress: :8554
rtpAddress: :8000
rtcpAddress: :8001

rtmpAddress: ""
hlsAddress: ""
webrtcAddress: ""
apiAddress: ""
metricsAddress: ""
webrtc: no

paths:
  all:
    source: publisher
```

Se puede observar que, como solo se va a usar el protocolo RTSP, se han establecido como vacíos el resto para evitar conflictos de puertos en la computadora.



```
mediamtx ./mediamtx.yml
> mediamtx ./mediamtx.yml
2026/01/07 12:39:44 INF MediaMTX v1.15.5
2026/01/07 12:39:44 INF configuration loaded from /home/helguera/Documents/REPOS/UNIR/master-ia/mediamtx1/mediamtx.yml
2026/01/07 12:39:44 INF [RTSP] listener opened on :8554 (TCP), :8000 (UDP/RTP), :8001 (UDP/RTCP)
2026/01/07 12:39:44 INF [RTMP] listener opened on
2026/01/07 12:39:44 INF [HLS] listener opened on
2026/01/07 12:39:44 INF [SRT] listener opened on :8890 (UDP)
```

Ilustración 6. Servidor MediamMTX corriendo

La Ilustración 6 muestra el resultado de lanzar el servidor con el fichero de configuraciones indicado previamente. Se puede observar que se queda a la espera de un flujo de datos por el puerto TCP 8554. Aquí es donde entra en juego FFmpeg, mediante el siguiente comando, se desviará el flujo de video de la cámara al servidor RTSP:

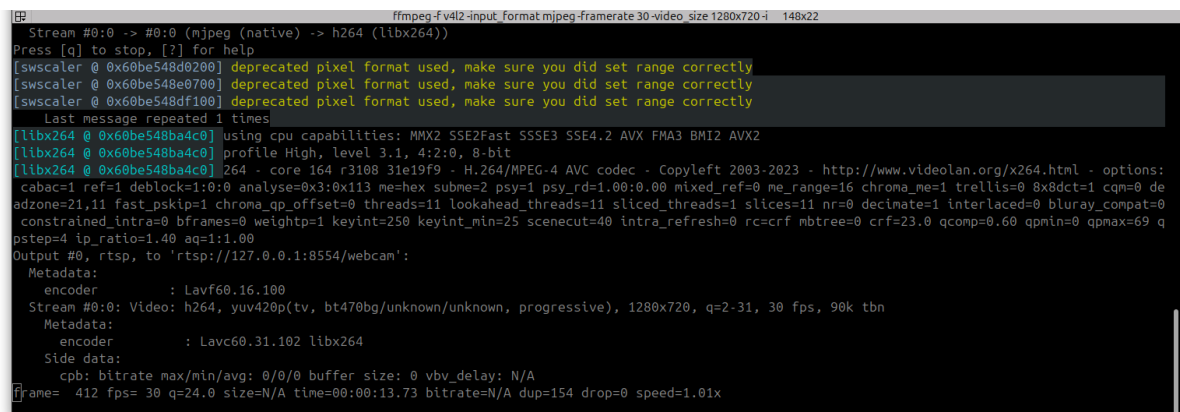
```
ffmpeg \
-f v4l2 -input_format mjpeg -framerate 30 -video_size 1280x720 -i /dev/video0 \
-vcodec libx264 -preset veryfast -tune zerolatency -pix_fmt yuv420p \
-f rtsp rtsp://127.0.0.1:8554/webcam
```

Es importante entender los parámetros que se incluyen en el comando:

- Captura de vídeo:
 - `-f v4l2`: Especifica que la fuente de entrada utiliza la interfaz Video4Linux2 (V4L2), estándar para dispositivos de captura de vídeo en sistemas Linux.
 - `-input_format mjpeg`: Indica que la cámara entrega el vídeo en formato Motion JPEG (MJPEG), lo que reduce la carga de procesamiento en la captura y mejora la estabilidad en webcams comunes.
 - `-framerate 30`: Fija la frecuencia de captura a 30 fotogramas por segundo, un valor habitual para vídeo en tiempo real.
 - `-video_size 1280x720`: Define la resolución de captura como 1280x720 píxeles (HD).
 - `-i /dev/video0`: Selecciona el dispositivo de captura correspondiente a la webcam, identificado en Linux como `/dev/video0`.
- Codificación de vídeo (procesamiento):

- -vcodec libx264: Utiliza el códec H.264/AVC mediante la librería libx264, ampliamente soportado y eficiente para streaming.
- -preset veryfast: Prioriza la velocidad de codificación sobre la eficiencia de compresión, reduciendo el uso de CPU y la latencia.
- -tune zerolatency: Ajusta el codificador para minimizar la latencia extremo a extremo, deshabilitando mecanismos como el lookahead y los B-frames.
- -pix_fmt yuv420p: Fuerza el formato de píxel YUV 4:2:0, que es el más compatible con servidores RTSP y clientes multimedia.
- Publicación de flujo (salida):
 - -f rtsp: Especifica que el flujo de salida se encapsula usando el protocolo RTSP, empleado para el control y la transmisión de contenido multimedia en tiempo real.
 - rtsp://127.0.0.1/webcam: Define la URL RTSP del servidor de destino (en este caso, MediaMTX ejecutándose en localhost), así como el path (/webcam) bajo el cual se publica el flujo.

La Ilustración 7 muestra el resultado de la consola cuando se lanza el comando FFmpeg.



```
Stream #0:0 -> #0:0 (mjpeg (native) -> h264 (libx264))
Press [q] to stop, [?] for help
[swscaler @ 0x60be548d0200] deprecated pixel format used, make sure you did set range correctly
[swscaler @ 0x60be548e0700] deprecated pixel format used, make sure you did set range correctly
[swscaler @ 0x60be548df100] deprecated pixel format used, make sure you did set range correctly
Last message repeated 1 times
[libx264 @ 0x60be548ba4c0] using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX FMA3 BMI2 AVX2
[libx264 @ 0x60be548ba4c0] profile High, level 3.1, 4:2:0, 8-bit
[libx264 @ 0x60be548ba4c0] 264 - core 164 r3108 31e19f9 - H.264/MPEG-4 AVC codec - Copyleft 2003-2023 - http://www.videolan.org/x264.html - options:
cabac=1 ref=1 deblock=1:0:0 analyse=0x3:0x113 me=hex subme=2 psy=1 psy_rd=1.00:0.00 mixed_ref=0 me_range=16 chroma_me=1 trellis=0 8x8dct=1 cqm=0 de
adzone=21,11 fast_pskip=1 chroma_qp_offset=0 threads=11 lookahead_threads=11 sliced_threads=11 nr=0 decimate=1 interlaced=0 bluray_compat=0
constrained_intra=0 bframes=0 weightp=1 keyint=250 keyint_min=25 scenecut=40 intra_refresh=0 rc=crf mbtree=0 crf=23.0 qcomp=0.60 qpmin=0 qpmax=69 q
pstep=4 ip_ratio=1.40 aq=1:1.00
Output #0, rtsp, to 'rtsp://127.0.0.1:8554/webcam':
  Metadata:
    encoder      : Lavf60.16.100
  Stream #0:0 Video: h264, yuv420p(tv, bt470bg/unknown/unknown, progressive), 1280x720, q=2-31, 30 fps, 90k tbn
  Metadata:
    encoder      : Lavc60.31.102 libx264
  Side data:
    cpb: bitrate max/min/avg: 0/0/0 buffer size: 0 vbv_delay: N/A
frame= 412 fps= 30 q=24.0 size=N/A time=00:00:13.73 bitrate=N/A dup=154 drop=0 speed=1.01x
```

Ilustración 7. FFmpeg enviando el flujo de video al servidor RTSP MediaMTX

Si se accede a la URL que se ha establecido previamente (`rtsp://127.0.0.1:8554/webcam`) desde un navegador web, una aplicación de vídeo se abrirá mostrando el contenido en directo de lo que la cámara está viendo (Ilustración 8):

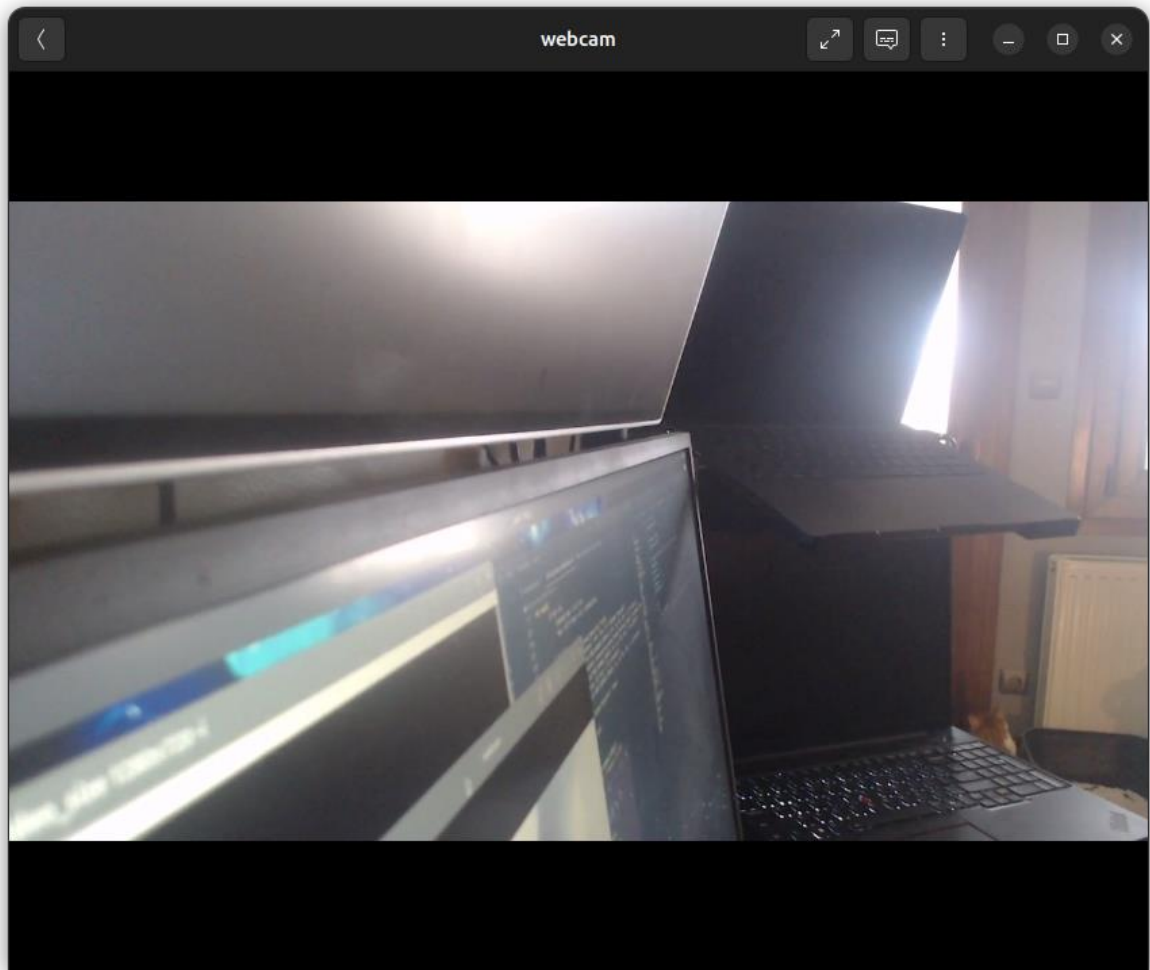


Ilustración 8. Ejemplo del flujo de video de la webcam USB mediante el protocolo RTSP

En este punto, ya se cuenta con un laboratorio con cámaras web normales que funcionan con el protocolo RTSP, el mismo que se usará en las cámaras de vigilancia reales en la solución final.

Para el desarrollo de la aplicación se necesita usar varias cámaras al mismo tiempo, lo cual no es un problema para la solución propuesta previamente. Simplemente, es necesario utilizar puertos diferentes para cada cámara. Esto se puede hacer gracias al fichero `mediamtx.yml`, el cual indica los puertos a utilizar.

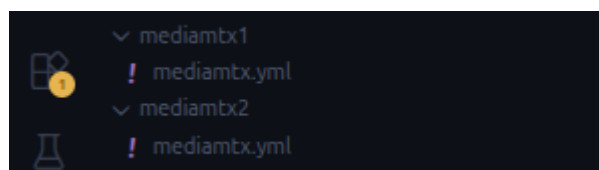


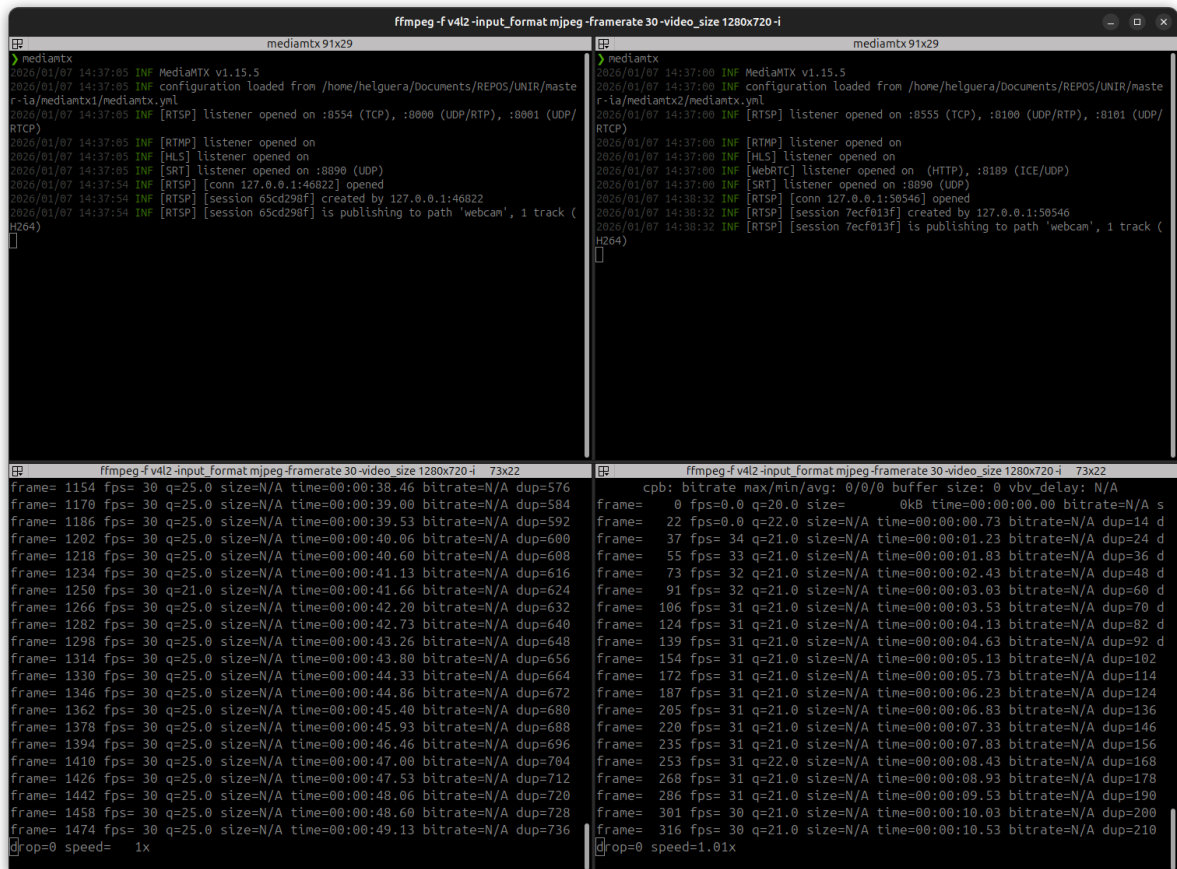
Ilustración 9. Estructura de ficheros de configuración para múltiples servidores RTSP

En el ejemplo se ha utilizado el puerto 8555 y se le ha enviado el flujo de vídeo de la cámara /dev/video0. Por lo tanto, siguiendo la estructura de la Ilustración 9, se puede lanzar múltiples servidores RTSP a los que se les enviará los diferentes flujos de vídeo de las cámaras disponibles.

```
rtspAddress: :8555
rtpAddress: :8100
rtcpAddress: :8101

rtmpAddress: ""
hlsAddress: ""
webrtcAddress: ""
apiAddress: ""
metricsAddress: ""
webrtc: no

paths:
  all:
    source: publisher
```



The image shows two terminal windows side-by-side, both displaying the output of an ffmpeg command: `ffmpeg -f v4l2 -input_format mjpeg -framerate 30 -video_size 1280x720 -i`. The left window shows the output for a stream from `mediatx91x29`, and the right window shows the output for a stream from `mediatx91x29`. Both windows show a list of frames with their respective FPS, QP, size, time, bitrate, and duplication status. The left window shows frames from 1154 to 1474, and the right window shows frames from 22 to 316. The output indicates that the streams are being processed and outputted as H264.

Ilustración 10. Ejemplo de dos cámaras emitiendo flujo de vídeo por RTSP simultáneamente

La Ilustración 10 muestra dos servidores RTSP recibiendo dos flujos de vídeo diferentes de dos cámaras simultáneamente, lo que genera dos URLs diferentes:

- rtsp://127.0.0.1:8554
- rtsp://127.0.0.1:8555

5.3.4. Visualizar el flujo de vídeo RTSP con Python

El uso de RTSP permite desacoplar el sistema de análisis (que se ejecuta en un equipo de recursos limitados) del dispositivo de captura, delegando en la cámara la codificación del vídeo (normalmente H.264/H.265) y recibiendo en el sistema local un flujo ya comprimido. De esta forma, el procesamiento del sistema se centra en la decodificación y en el análisis mediante visión por computador, evitando dependencias físicas como la conexión USB.

5.3.4.1. Selección del backend de decodificación (FFmpeg/GStreamer)

La captura se implementa mediante OpenCV (`cv2.VideoCapture`), que permite abrir flujos RTSP a través de distintos backends. En este proyecto se prioriza FFmpeg por su compatibilidad general con RTSP, y se incluye un fallback con GStreamer para entornos Linux donde la estabilidad o la latencia pueden beneficiarse de un pipeline explícito.

La apertura del stream se plantea como un procedimiento robusto: primero se intenta con FFmpeg y, si no se logra abrir el flujo, se intenta con GStreamer:

```
cap = cv2.VideoCapture(rtsp_url, cv2.CAP_FFMPEG)
if not cap.isOpened():
    gst = build_gstreamer_pipeline(rtsp_url, latency_ms=100)
    cap = cv2.VideoCapture(gst, cv2.CAP_GSTREAMER)
```

5.3.4.2. Configuración de baja latencia (GStreamer)

En el caso de GStreamer, se define un pipeline orientado a baja latencia: la fuente RTSP se configura con un parámetro `latency`, se depaquetiza el RTP, se parsea y decodifica el stream H.264, y finalmente se entrega a OpenCV mediante `appsink`. Además, se ajustan opciones como `drop=true` y `sync=false` para priorizar el frame más reciente frente a la reproducción “en tiempo” del stream, lo cual es útil cuando el sistema no puede procesar todos los frames entrantes.


```
def build_gstreamer_pipeline(rtsp_url, latency_ms=100):  
    return (  
        f"rtspsrc location={rtsp_url} latency={latency_ms} ! "  
        "rtph264depay ! h264parse ! avdec_h264 ! "  
        "videoconvert ! appsink drop=true sync=false"  
    )
```

En sistemas de bajos recursos, este tipo de configuración resulta crítica para evitar que el análisis se realice sobre frames antiguos, fenómeno que produce un retraso acumulativo y degrada la utilidad de un sistema de vigilancia en tiempo real.

5.3.4.3. Minimización del buffering y control de rendimiento

Una vez abierta la captura, se intenta reducir la cola interna de frames mediante el ajuste del tamaño de buffer. Aunque su efectividad depende del backend y del sistema, constituye una medida complementaria para reducir la latencia:

```
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
```

El vídeo se procesa como una secuencia de fotogramas obtenidos de forma iterativa. En cada iteración se lee un frame, se valida y se aplica un preprocesado mínimo. El redimensionado es una optimización explícita para asegurar viabilidad en hardware modesto: reduce el coste del procesamiento posterior (detección de personas) sin necesidad de reducir la funcionalidad del sistema.

```
ok, frame = cap.read()  
if ok and frame is not None:  
    h, w = frame.shape[:2]  
    scale = target_width / float(w)  
    frame = cv2.resize(frame, (target_width, int(h * scale)))
```

Para monitorizar el impacto de estos ajustes, se calcula la tasa de frames por segundo (FPS) en tiempo real. Esta métrica permite validar la capacidad del sistema para procesar el stream de forma sostenida, y facilita la toma de decisiones posteriores, por ejemplo, respecto a la frecuencia de inferencia de modelos o la selección de substreams de menor resolución.

```
dt = time.time() - prev_time  
fps = 0.9 * fps + 0.1 * (1.0 / dt)
```

5.3.4.4. Manejo de fallos y reconexión automática

En un escenario real, el stream RTSP puede fallar por cortes de red, reinicios de la cámara o pérdidas temporales de conectividad. Por ello, la obtención del flujo de video se ha diseñado con tolerancia a fallos: si `read()` devuelve un frame inválido, se libera el recurso y se reintenta la conexión tras un breve intervalo. Este patrón evita bloqueos y permite que el sistema recupere su funcionamiento sin intervención manual.

```
ok, frame = cap.read()
if not ok or frame is None:
    cap.release()
    cap = None
    time.sleep(reconnect_wait_s)
    continue
```

El código completo de este script se puede encontrar en el Anexo A. Es la base para las siguientes fases del proyecto.

5.4.DETECCIÓN DE PERSONAS MEDIANTE MODELOS DE APRENDIZAJE PROFUNDO

La detección automática de personas constituye un problema clásico dentro del ámbito de la visión por computador basada en aprendizaje profundo, y representa uno de los pilares fundamentales del sistema desarrollado. En el contexto de la vigilancia doméstica, este módulo actúa como un filtro semántico de alto nivel, permitiendo discriminar entre estímulos visuales irrelevantes y la presencia real de individuos en la escena.

Desde el punto de vista formal, la detección de personas se plantea como un problema de detección de objetos multiclase, en el que el objetivo es identificar instancias de una clase concreta (person) dentro de una imagen, proporcionando tanto su localización espacial mediante bounding boxes como una estimación probabilística de confianza asociada a cada detección.

5.4.1. Selección del modelo: MobileNet-SSD

Para la implementación de la detección de personas se ha seleccionado el modelo MobileNet-SSD, que combina la arquitectura Single Shot Detector (SSD) con una red base ligera del tipo MobileNet. Esta elección responde a un compromiso explícito entre precisión y eficiencia computacional.

El enfoque SSD permite realizar la detección de objetos en una única pasada (single shot) por la red neuronal, evitando etapas intermedias de generación de propuestas de regiones (region proposals), lo que reduce significativamente la latencia respecto a modelos de dos etapas como Faster R-CNN (Liu W. , y otros, 2016). Este aspecto resulta clave para aplicaciones en tiempo real y en el código se ve re

Por su parte, MobileNet introduce el uso de convoluciones separables en profundidad (depthwise separable convolutions), una técnica que descompone la convolución estándar en dos operaciones más simples, reduciendo drásticamente el número de parámetros y operaciones aritméticas necesarias (Howard, y otros, 2017). Gracias a ello, MobileNet-SSD se adapta especialmente bien a sistemas embebidos o equipos de bajo consumo, alineándose con los objetivos de este proyecto.

5.4.2. Uso de modelos preentrenados y transferencia de conocimiento

El modelo empleado ha sido previamente entrenado sobre el conjunto de datos PASCAL VOC, uno de los benchmarks clásicos en detección de objetos, que incluye la clase person entre sus categorías principales (Everingham, van Gool, Williams, Winn, & Zisserman, 2012). El uso de modelos preentrenados constituye una forma de transfer learning, en la que el conocimiento adquirido durante el entrenamiento en grandes conjuntos de datos se reutiliza para resolver tareas específicas sin necesidad de entrenar una red desde cero.

Este enfoque reduce considerablemente los requisitos computacionales y de datos, y resulta especialmente adecuado para proyectos aplicados como el presente trabajo (Jialin Pan & Yang, 2009).

5.4.3. Implementación del módulo de detección de personas

Una vez justificada la elección del modelo MobileNet-SSD y el uso de pesos preentrenados, el siguiente paso consistió en integrar el detector dentro del flujo completo del sistema de vigilancia: recepción del vídeo (RTSP), preprocesado, inferencia, filtrado de detecciones y generación de eventos. El objetivo no era únicamente “detectar personas”, sino hacerlo de forma estable en tiempo real, en hardware modesto y en condiciones domésticas reales (variaciones de luz, cámaras con compresión H.264, movimiento de mascotas, etc.).

En esta fase se priorizó un enfoque incremental: primero se buscó que el modelo funcionara correctamente sobre frames estáticos; después se integró con vídeo RTSP; finalmente se

incorporaron mecanismos de robustez (reconexión, confirmación temporal y cooldown) para reducir falsas alarmas y evitar eventos redundantes. Esta forma de trabajo permitió aislar problemas, medir rendimiento y tomar decisiones técnicas justificables dentro del marco del TFM.

El sistema utiliza un modelo preentrenado en formato Caffe, compuesto por dos ficheros: el grafo/arquitectura (.prototxt) y los pesos (.caffemodel). La carga se realiza una vez al inicio para evitar sobrecostes en tiempo de ejecución:

```
net = cv2.dnn.readNetFromCaffe(prototxt_path, model_path)
```

Ambos ficheros se pueden obtener mediante wget del repositorio oficial de GitHub:

```
wget https://raw.githubusercontent.com/chuanqi305/MobileNet-SSD/master/deploy.prototxt -O MobileNetSSD_deploy.prototxt  
  
wget https://github.com/chuanqi305/MobileNet-SSD/raw/master/mobilenet_iter_73000.caffemodel -O  
MobileNetSSD_deploy.caffemodel
```

Tienen que estar localizados en una carpeta dentro del proyecto para ser accedidos fácilmente, como lo muestra la Ilustración 11.

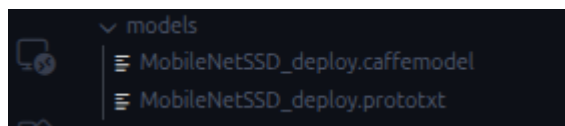


Ilustración 11. Localización de los ficheros necesarios para MobileNetSSD en el proyecto

A partir de este momento, net se reutiliza en cada iteración del bucle, ejecutando inferencias sobre cada frame.

El sistema recibe el vídeo a través de un flujo RTSP, que se abre dinámicamente utilizando el backend más adecuado disponible en el sistema, como se ha explicado en el apartado 2.2.4.

Una decisión clave fue introducir target_width como parámetro configurable. Aunque la red MobileNet-SSD internamente trabaja con 300×300, el frame real se utiliza para overlays, visualización y guardado de evidencias. Si se mantiene la resolución nativa (por ejemplo, 1920×1080), el sistema sufre penalizaciones en copia y manipulación de memoria, dibujo de bounding boxes, escritura a disco y renderizado en pantalla. Por ello se optó por redimensionar manteniendo la proporción:

```
h, w = frame.shape[:2]
if target_width and w != target_width:
    scale = target_width / float(w)
    frame = cv2.resize(frame, (target_width, int(h * scale)))
```

En pruebas comparativas realizadas sobre la misma escena, cámara y CPU, se observó que pasar de 1080p a un ancho objetivo de 800 mejoraba la estabilidad del FPS y reducía picos de latencia. En equipos modestos, el efecto fue especialmente notable cuando se usaban múltiples cámaras conectadas al mismo sistema.

La función `detect_person()` encapsula el núcleo de IA. En lugar de utilizar un framework de entrenamiento completo, se emplea OpenCV DNN para simplificar el despliegue y mantener el entorno ligero. El preprocesado se realiza mediante `blobFromImage`, produciendo un tensor compatible con MobileNet-SSD:

```
blob = cv2.dnn.blobFromImage(
    cv2.resize(frame, (300, 300)),
    scalefactor=0.007843,
    size=(300, 300),
    mean=127.5
)
net.setInput(blob)
detections = net.forward()
```

Esta configuración se mantuvo tras comparar variantes con y sin normalización. La entrada sin normalizar produjo detecciones inconsistentes y mayor sensibilidad a la iluminación, especialmente en interiores con luz cálida, por lo que se adoptó el preprocesado estándar recomendado por el modelo.

El modelo devuelve detecciones multiclase, pero el sistema se limita a la detección de personas, aplicando un filtrado semántico:

```
if CLASSES[class_id] != "person":
    continue
```

y un filtrado por confianza:

```
if confidence < conf_thresh:
    continue
```

Este umbral se ajustó mediante pruebas iterativas en escenarios domésticos, incluyendo escenas vacías con cambios de luz, paso de mascotas, personas parcialmente visibles y

oclusiones. Con valores bajos se detectaron falsos positivos, especialmente con sombras y reflejos. Al aumentar el umbral a 0.55 se redujeron estos casos sin perder detecciones consistentes, por lo que se adoptó como valor por defecto, manteniéndose configurable.

Las coordenadas devueltas por SSD están normalizadas y se convierten a píxeles del frame mediante:

```
box = detections[0, 0, i, 3:7] * [w, h, w, h]
(startX, startY, endX, endY) = box.astype("int")
```

La visualización de bounding boxes y confianzas fue clave para la validación, permitiendo detectar problemas de configuración, umbrales inadecuados o errores de resolución durante las pruebas.

Se observó que podían producirse detecciones puntuales en un único frame debido a ruido de compresión o cambios bruscos de iluminación. Para evitar generar alertas con detecciones efímeras se implementó un mecanismo de confirmación temporal:

```
if person_found:
    consecutive_person_frames += 1
else:
    consecutive_person_frames = 0
```

Solo cuando consecutive_person_frames alcanza confirm_frames se considera una detección confirmada. En pruebas domésticas, confirm_frames=5 resultó un buen compromiso entre robustez y latencia, descartando falsos positivos aislados sin introducir una demora excesiva.

Otro problema observado fue la generación repetitiva de eventos cuando una persona permanecía en escena. Para evitarlo se introdujo un periodo de cooldown:

```
in_cooldown = (now - last_alert_time) < cooldown_s
```

Con cooldown_s=20 se redujo la repetición de alertas y se mantuvo un registro limpio, de modo que el sistema actúa como detector de incidentes y no como un contador de frames.

Finalmente, se implementó un cálculo de FPS suavizado para monitorizar el rendimiento real y detectar degradaciones progresivas:

```
instant_fps = 1.0 / dt
fps = 0.9 * fps + 0.1 * instant_fps
```

El suavizado evita fluctuaciones bruscas y permite observar tendencias a lo largo del tiempo. Esta métrica resultó útil para ajustar `target_width` y los umbrales, garantizando un rendimiento estable durante sesiones prolongadas.

El código completo del script explicado en este apartado se puede encontrar en el **Error! Reference source not found..**

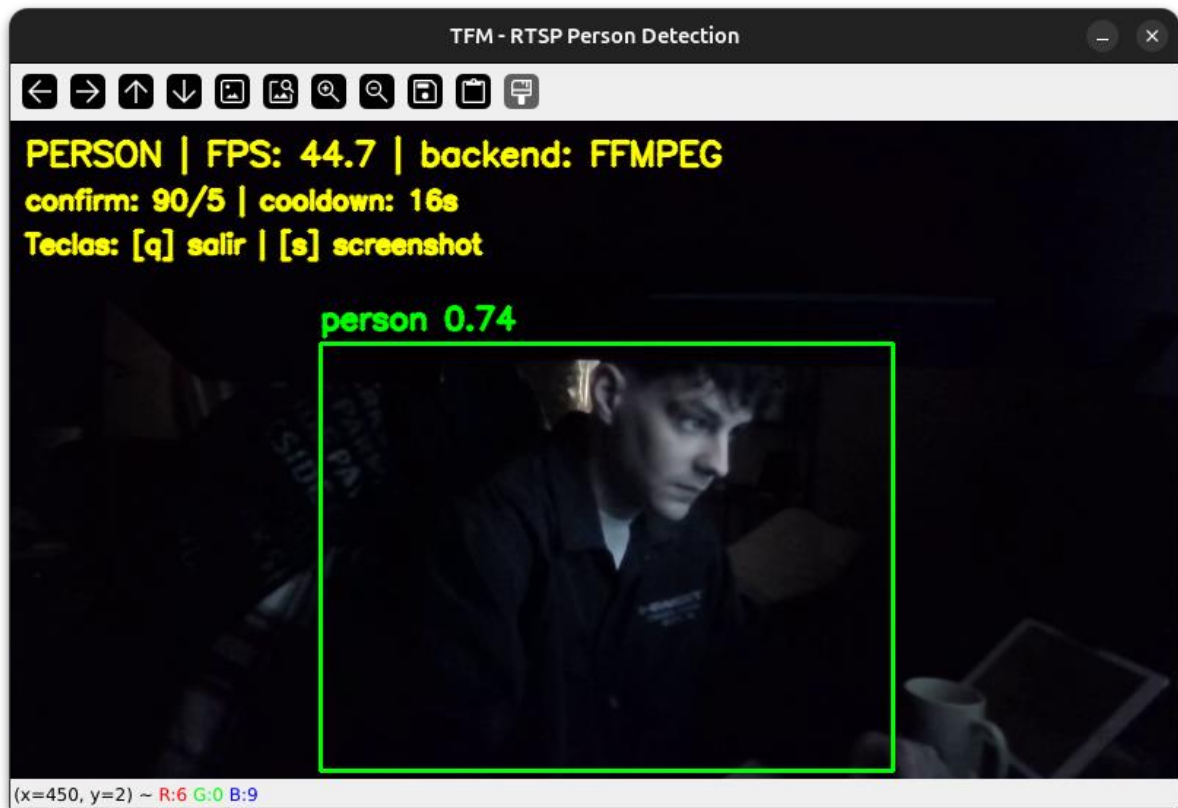


Ilustración 12. Ejemplo de detección de una persona en condiciones de poca luz

La Ilustración 12 muestra el script en acción. El modelo ha detectado una persona en la imagen y ha dibujado el bounding box. Incluso en una situación de baja luminosidad, el modelo ha sido capaz de detectar una persona con una confianza del 74%. En la esquina superior izquierda, el script muestra información importante como el tipo de objeto que se está detectando, los FPS del flujo de vídeo y el backend que se está utilizando. También se puede observar que hay un cooldown de 20 segundos. Como se ha explicado previamente, cuando se detecta una persona, se espera una cantidad X de segundos antes de volver a detectarla para evitar un exceso de alertas innecesarias.

5.5. CREACIÓN DEL DASHBOARD

Todo lo que se ha explicado hasta este punto en el documento ha sido mediante scripts individuales, pero, como se indicó en los objetivos, el proyecto consiste en una aplicación completa, donde todas las piezas individuales funcionen juntas.

5.5.1. Funcionalidad del dashboard

El sistema incorpora un dashboard web que actúa como interfaz principal de interacción con la aplicación. A través de este panel, el usuario puede gestionar los distintos elementos del sistema de vigilancia de forma centralizada y sencilla.

El dashboard permite la gestión de usuarios, incluyendo el registro y la administración de cuentas, así como la gestión de cámaras de vigilancia, posibilitando su alta, modificación y eliminación. Además, ofrece la visualización en tiempo real de los flujos de vídeo asociados a las cámaras registradas.

Asimismo, el sistema proporciona un apartado de alertas donde se muestran los eventos detectados, junto con una captura de imagen asociada a cada alerta como evidencia visual. Estas funcionalidades permiten supervisar el estado del sistema y revisar de forma rápida los eventos relevantes detectados.

Los apartados siguientes describen con mayor detalle el funcionamiento de cada una de estas capacidades.

5.5.2. Elección del Backend y Frontend: FastAPI

Para la implementación de la capa de gestión del sistema se ha optado por utilizar FastAPI como base tanto del backend como del dashboard web de administración. Esta elección responde a criterios de simplicidad, eficiencia y coherencia tecnológica con el resto del proyecto, además de encajar de forma natural en una arquitectura orientada al procesamiento local y a la integración de módulos de visión por computador.

FastAPI es un framework moderno para el desarrollo de aplicaciones web y APIs en Python, diseñado para ofrecer un alto rendimiento con una estructura clara y ligera. Su enfoque asíncrono y su compatibilidad nativa con estándares como ASGI permiten gestionar múltiples peticiones concurrentes de forma eficiente, lo que resulta especialmente relevante en un

sistema que debe coordinar la captura de vídeo, la gestión de alertas y la interacción con el usuario sin introducir latencias innecesarias (Ramírez, 2025).

Desde el punto de vista del desarrollo, el uso de FastAPI facilita la creación de una única base de código que actúa simultáneamente como API y como interfaz web. Esta unificación reduce la complejidad del sistema, evita duplicidades y simplifica el mantenimiento, ya que la lógica de negocio, la gestión de estados y el acceso a los datos se concentran en un único servicio. Además, el uso de Python como lenguaje común permite una integración directa con los módulos de visión por computador y aprendizaje profundo utilizados en el proyecto, sin necesidad de capas de comunicación adicionales.

5.5.3. Endpoints del Backend

En este apartado se recogen los endpoints más relevantes de la API desarrollada, organizados por módulos funcionales. Su objetivo es facilitar la interacción entre el backend y el dashboard web, permitiendo gestionar usuarios, cámaras y alertas. El código completo y la especificación detallada pueden consultarse en el repositorio de GitHub (<https://github.com/Helguera/TFM-AI-UNIR-2026>).

5.5.3.1. Módulo de autenticación de usuarios

Este módulo implementa la gestión de acceso al sistema mediante autenticación basada en JWT y cookies seguras:

- **POST** /api/auth/register: permite registrar nuevos usuarios verificando previamente que no existan credenciales duplicadas en la base de datos.
- **POST** /api/auth/login: realiza la autenticación mediante OAuth2, generando un token JWT que se almacena en una cookie HTTP-only para su uso seguro en el dashboard.
- **POST** /api/auth/logout: finaliza la sesión eliminando la cookie de autenticación.
- **GET** /api/auth/me: devuelve la información del usuario autenticado, utilizada por la interfaz para comprobar el estado de sesión.

5.5.3.2. Módulo de gestión de cámaras

Este bloque permite administrar las cámaras IP registradas y controlar el proceso de detección asociado:

- **GET** /api/cameras/: obtiene el listado de cámaras del usuario junto con su estado operativo.
- **POST** /api/cameras/: registra una nueva cámara con su URL RTSP y parámetros de detección configurables.
- **PUT** /api/cameras/{camera_id} y **DELETE** /api/cameras/{camera_id}: permiten modificar o eliminar cámaras existentes.
- **POST** /api/cameras/{camera_id}/start y /stop: inician o detienen el proceso de análisis de vídeo.
- **GET** /api/cameras/{camera_id}/stream: ofrece visualización en directo mediante streaming MJPEG integrado en el dashboard.

5.5.3.3. Módulo de alertas y eventos

Este módulo gestiona los eventos generados cuando se detecta una persona en el sistema:

- **GET** /api/alerts/: devuelve el historial de alertas del usuario, con opciones de filtrado y paginación.
- **GET** /api/alerts/stats: proporciona estadísticas agregadas (alertas totales, pendientes, generadas hoy).
- **PUT** /api/alerts/{alert_id}/read: marca alertas como vistas desde el dashboard.
- **GET** /api/alerts/{alert_id}/image: permite acceder a la evidencia visual asociada a cada evento.

5.5.3.4. Módulo de configuración de notificaciones Telegram

Este módulo permite gestionar la integración opcional con Telegram como canal externo de alertas. A través de estos endpoints, el usuario puede configurar el bot de notificación desde el dashboard, habilitar o deshabilitar el envío de mensajes y verificar el estado del servicio.

Los principales endpoints expuestos son:

- **GET** /api/telegram/config: obtiene la configuración actual de Telegram asociada al usuario autenticado, devolviendo parámetros como el chat ID y el estado de activación.

- **POST** /api/telegram/config: crea o registra una nueva configuración del bot, almacenando de forma persistente las credenciales necesarias para el envío de notificaciones.
- **PUT** /api/telegram/config: permite actualizar parcialmente la configuración existente, incluyendo la activación o desactivación del servicio.
- **DELETE** /api/telegram/config: elimina la configuración almacenada y detiene el servicio de notificaciones asociado.
- **POST** /api/telegram/test: realiza una prueba de conexión con las credenciales proporcionadas, permitiendo validar que el bot puede enviar mensajes correctamente.
- **GET** /api/telegram/status: devuelve un resumen del estado actual del sistema Telegram, indicando si está configurado, habilitado y en ejecución.

5.5.4. Integración de los módulos de visión artificial

El módulo de detección de personas se ha diseñado como un servicio independiente dentro de la aplicación, desacoplado del dashboard y de la lógica de presentación (Ilustración 13). Esta decisión permite que el análisis de vídeo y la gestión de cámaras se mantengan como una capa autónoma, mientras que el dashboard actúa únicamente como interfaz de control y visualización. De este modo, el sistema evita dependencias directas entre la lógica de visión por computador y la capa web, facilitando tanto el mantenimiento como la evolución futura de la aplicación.

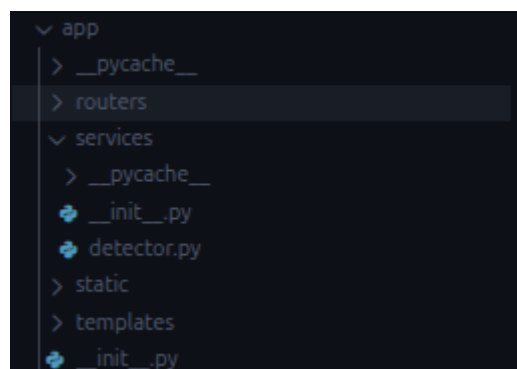


Ilustración 13. Estructura del proyecto con el módulo independiente de detección de personas

La integración entre el módulo de detección y el resto del sistema se realiza a través de una interfaz bien definida, gestionada por el componente DetectionManager.

```
class DetectionManager:
    """
    Singleton manager for all camera workers.
    """
    _instance = None
    _lock = threading.Lock()
```

Este gestor actúa como punto central de coordinación, encargándose de iniciar y detener los procesos de detección asociados a cada cámara, así como de exponer información de estado y frames recientes cuando son solicitados por la API. El dashboard interactúa con este gestor únicamente mediante llamadas de alto nivel, sin conocer los detalles internos de captura o procesamiento del vídeo.

```
# Global instance
detection_manager = DetectionManager()
```

Cada cámara activa se gestiona mediante un hilo independiente (CameraWorker), lo que permite ejecutar la detección de forma concurrente sobre múltiples flujos RTSP.

```
class CameraWorker(threading.Thread):
    """
    Thread per camera: RTSP capture + detection + events.
    Does NOT use imshow/waitKey. Only updates the last processed frame.
    """
```

Esta separación garantiza que un fallo, reconexión o degradación en una cámara no afecte al resto del sistema ni bloquee el funcionamiento del dashboard. Desde el punto de vista de integración, el dashboard solo necesita conocer si una cámara está en ejecución, su estado actual y, opcionalmente, el último frame disponible para visualización en directo.

La comunicación entre el módulo de detección y el sistema de alertas se realiza mediante un mecanismo de callback.

```
def create_alert_callback():
    """Create callback function for detection alerts that saves to
    database."""
    def save_alert(camera_id: int, event_type: str, confidence: float, detail:
    str, image_path: str):
        db = SessionLocal()
        try:
            alert = Alert(
                camera_id=camera_id,
                event_type=event_type,
                confidence=confidence,
                detail=detail,
                image_path=image_path
            )
            db.add(alert)
            db.commit()
        except Exception as e:
            print(f"Error saving alert: {e}")
            db.rollback()
        finally:
            db.close()
    return save_alert
```

Cuando se detecta un evento relevante, el módulo de detección invoca una función registrada previamente, delegando la persistencia del evento y la notificación al resto de la aplicación. Este enfoque evita dependencias circulares y permite que el módulo de detección no tenga conocimiento de cómo se almacenan las alertas ni de cómo se notifican al usuario, reforzando la separación de responsabilidades.

Desde el punto de vista arquitectónico, este diseño modular permite incorporar nuevos tipos de análisis sin modificar la estructura del dashboard. En el futuro podrían añadirse módulos adicionales, por ejemplo, reconocimiento facial, detección de objetos específicos o análisis de comportamiento reutilizando el mismo patrón de integración: procesos independientes gestionados por un controlador central y comunicados mediante interfaces y callbacks.

5.5.5. Persistencia de datos

Para la persistencia de datos del sistema se ha optado por utilizar una base de datos relacional ligera, concretamente SQLite (SQLite, 2025), integrada a través de un ORM en Python. Esta elección responde principalmente a los requisitos del proyecto, orientado a la ejecución en sistemas de bajos recursos y a un entorno doméstico donde se priorizan la simplicidad, la eficiencia y la facilidad de despliegue.

SQLite es una base de datos embebida que no requiere la ejecución de un servidor independiente, ya que se almacena directamente en un fichero local. Desde el punto de vista funcional, SQLite ofrece todas las capacidades necesarias para el sistema desarrollado, incluyendo soporte para transacciones, integridad referencial y consultas complejas. El volumen de datos gestionado, usuarios, cámaras y alertas, es moderado y no requiere las prestaciones de bases de datos orientadas a grandes cargas concurrentes. En este contexto, SQLite proporciona un rendimiento más que suficiente, con tiempos de acceso reducidos y una huella de memoria muy baja (SQLite, 2025).

Otro aspecto relevante es la portabilidad. Al estar basada en un único fichero, la base de datos puede copiarse, respaldarse o migrarse fácilmente entre sistemas, lo que resulta útil tanto durante la fase de desarrollo como en un posible despliegue real del sistema. Esta portabilidad facilita también la realización de pruebas y la recuperación del sistema en caso de fallo.

Cuenta con tres modelos, que corresponden a los tres grupos de endpoints explicados en la sección 5.5.3: alerts, cameras y users (Ilustración 14).

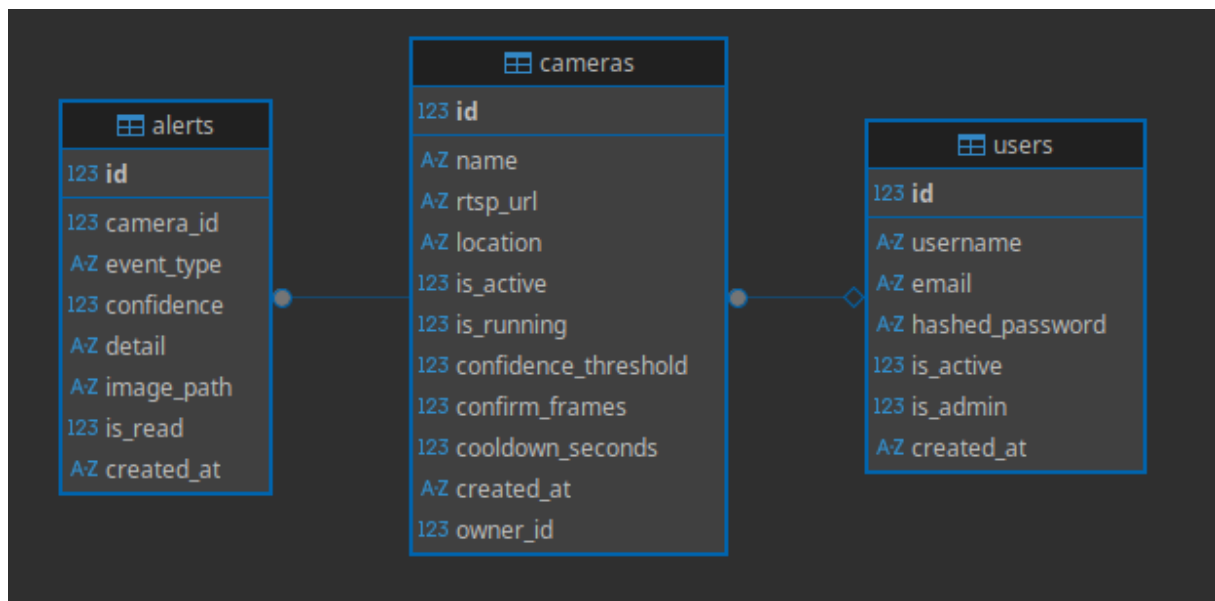


Ilustración 14. Modelos de la base de datos

5.5.5.1. Users

La entidad User representa a los usuarios registrados en el sistema y constituye la base del control de acceso y la personalización de la aplicación. Cada usuario se identifica de forma única mediante un identificador numérico y dispone de credenciales propias, compuestas por

un nombre de usuario y una dirección de correo electrónico, ambos definidos como únicos para evitar duplicidades.

Las credenciales de autenticación se almacenan de forma segura mediante el uso de contraseñas cifradas, garantizando que no se persisten contraseñas en texto plano. Adicionalmente, la entidad incluye campos que permiten gestionar el estado de la cuenta, como la activación o desactivación del usuario y la posibilidad de distinguir usuarios con privilegios administrativos.

La marca temporal de creación permite registrar el momento en el que se da de alta cada usuario. Desde el punto de vista relacional, cada usuario puede estar asociado a múltiples cámaras de vigilancia, estableciéndose una relación uno a muchos entre usuarios y cámaras. Esta relación permite que el sistema asigne de forma clara la propiedad de cada cámara y limite el acceso a los recursos en función del usuario autenticado.

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True, nullable=False)
    email = Column(String(100), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    is_active = Column(Boolean, default=True)
    is_admin = Column(Boolean, default=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    cameras = relationship("Camera", back_populates="owner")
```

5.5.5.2. Cameras

La entidad Camera representa los dispositivos de captura de vídeo registrados en el sistema y constituye el elemento central del proceso de vigilancia. Cada cámara se identifica mediante un identificador único y se define por un nombre descriptivo y la URL RTSP asociada al flujo de vídeo, que permite su integración con el módulo de detección.

La entidad incluye información opcional de localización, orientada a facilitar la identificación de la cámara por parte del usuario en el dashboard, así como campos que permiten gestionar su estado. En concreto, se distingue entre el estado de activación de la cámara y su estado de ejecución, lo que permite deshabilitar un dispositivo sin eliminarlo del sistema y controlar de forma independiente si el proceso de detección se encuentra en funcionamiento.

Además de la información básica, la entidad Camera incorpora parámetros de configuración específicos del proceso de detección, como el umbral de confianza, el número de frames necesarios para confirmar una detección y el tiempo de enfriamiento entre eventos consecutivos. Estos parámetros se almacenan a nivel de cámara, lo que permite adaptar el comportamiento del sistema a las características de cada entorno monitorizado.

La marca temporal de creación permite registrar cuándo se añadió la cámara al sistema. Desde el punto de vista relacional, cada cámara pertenece a un único usuario, estableciéndose una relación muchos a uno con la entidad User, lo que permite garantizar que cada dispositivo esté claramente asociado a su propietario. Asimismo, se define una relación uno a muchos con la entidad Alert, de modo que cada cámara puede generar múltiples alertas, las cuales se eliminan automáticamente si la cámara es borrada, manteniendo la coherencia del sistema.

```
class Camera(Base):
    __tablename__ = "cameras"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    rtsp_url = Column(String(500), nullable=False)
    location = Column(String(200), nullable=True)
    is_active = Column(Boolean, default=True)
    is_running = Column(Boolean, default=False)

    # Detection settings
    confidence_threshold = Column(Float, default=0.55)
    confirm_frames = Column(Integer, default=5)
    cooldown_seconds = Column(Integer, default=20)

    created_at = Column(DateTime, default=datetime.utcnow)
    owner_id = Column(Integer, ForeignKey("users.id"))

    owner = relationship("User", back_populates="cameras")
    alerts = relationship("Alert", back_populates="camera", cascade="all,
delete-orphan")
```

5.5.5.3. Alerts

La entidad Alert representa los eventos detectados por el sistema de vigilancia y constituye el mecanismo principal para registrar y gestionar las incidencias generadas a partir del análisis del vídeo. Cada alerta se identifica mediante un identificador único y está asociada obligatoriamente a una cámara concreta, lo que permite mantener la trazabilidad del evento y su origen.

La entidad incluye un campo que define el tipo de evento detectado, lo que permite diferenciar entre distintas clases de alertas, como intrusiones u otros eventos relevantes que puedan incorporarse en futuras extensiones del sistema. Adicionalmente, se almacena información cuantitativa opcional, como el nivel de confianza de la detección, así como un campo descriptivo que permite registrar detalles adicionales sobre el evento ocurrido.

Para cada alerta puede asociarse una evidencia visual mediante la ruta al archivo de imagen almacenado en el sistema. Este enfoque evita la persistencia directa de datos binarios en la base de datos, reduciendo su tamaño y manteniendo una separación clara entre los metadatos del evento y los recursos multimedia generados.

El campo que indica si la alerta ha sido leída permite gestionar el ciclo de vida del evento desde el punto de vista del usuario, diferenciando entre alertas pendientes de revisión y alertas ya gestionadas. La marca temporal de creación registra el momento exacto en el que se generó la alerta, facilitando su ordenación y análisis posterior.

Desde el punto de vista relacional, cada alerta pertenece a una única cámara, estableciéndose una relación muchos a uno con la entidad Camera. Esta relación permite agrupar las alertas por dispositivo y garantiza la coherencia del sistema al eliminar automáticamente las alertas asociadas cuando una cámara es eliminada.

```
class Alert(Base):
    __tablename__ = "alerts"

    id = Column(Integer, primary_key=True, index=True)
    camera_id = Column(Integer, ForeignKey("cameras.id"), nullable=False)
    event_type = Column(String(50), nullable=False)
    confidence = Column(Float, nullable=True)
    detail = Column(Text, nullable=True)
    image_path = Column(String(500), nullable=True)
    is_read = Column(Boolean, default=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    camera = relationship("Camera", back_populates="alerts")
```

5.5.6. Frontend

El frontend de la aplicación se ha desarrollado utilizando templates Ninja (FastAPI, 2025), integrados directamente en el backend basado en FastAPI. El uso de un backend y frontend unificados en una única aplicación FastAPI permite evitar la necesidad de desplegar servicios

adicionales, como servidores frontend independientes o frameworks JavaScript complejos. Este enfoque reduce el consumo de recursos, simplifica el proceso de despliegue y facilita el mantenimiento del sistema, aspectos especialmente relevantes en aplicaciones que deben ejecutarse de forma continua en hardware modesto.

Los templates Ninja proporcionan un mecanismo ligero para la generación de interfaces web a partir de plantillas HTML, permitiendo separar la lógica de presentación de la lógica de negocio sin introducir dependencias pesadas (GeeksForGeeks, 2025). Este enfoque resulta adecuado para dashboards de administración y monitorización, donde la prioridad es la claridad, la estabilidad y la rapidez de respuesta, más que la complejidad visual o la interacción avanzada del lado cliente.

Además, al compartir el mismo entorno tecnológico, el frontend puede interactuar directamente con los endpoints del backend sin capas intermedias, lo que mejora la coherencia del sistema y reduce posibles errores de integración. Esta integración directa facilita también el control de la autenticación y la gestión de sesiones, al reutilizar los mismos mecanismos de seguridad en toda la aplicación.

5.5.6.1. Login de usuario

La ventana de login (Ilustración 15) permite la conexión de un usuario al sistema. No se ha implementado un sistema de recuperación de contraseñas ya que se ha considerado que no entra en el alcance del proyecto.

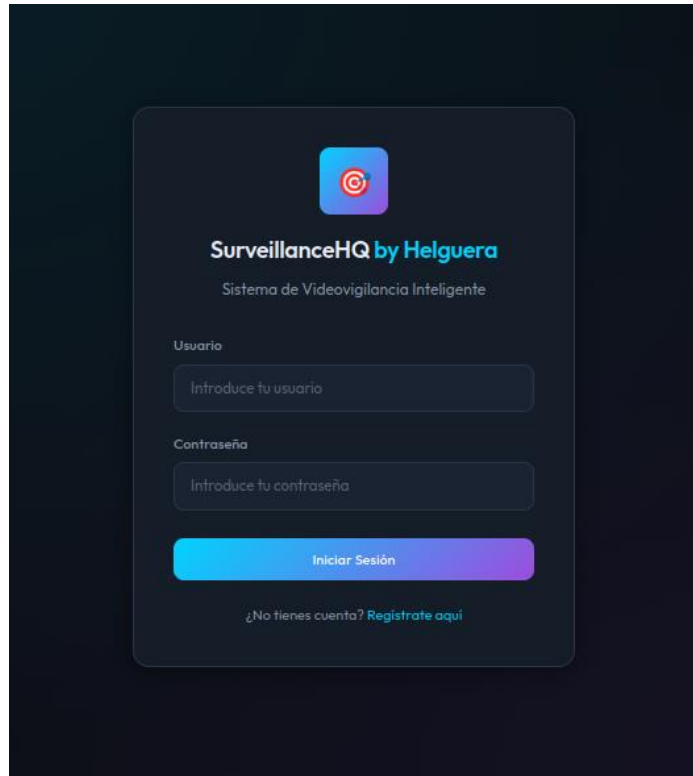


Ilustración 15. Ventana de login

5.5.6.2. Registro de usuario

La ventana de registro (Ilustración 16) permite crear un nuevo usuario en el sistema. Cada usuario tiene acceso las cámaras que ha añadido manualmente y no se pueden compartir entre usuarios.

Para identificar un usuario, se usa el id generado automáticamente, el email y el nombre de usuario. Los tres campos tienen que ser únicos.

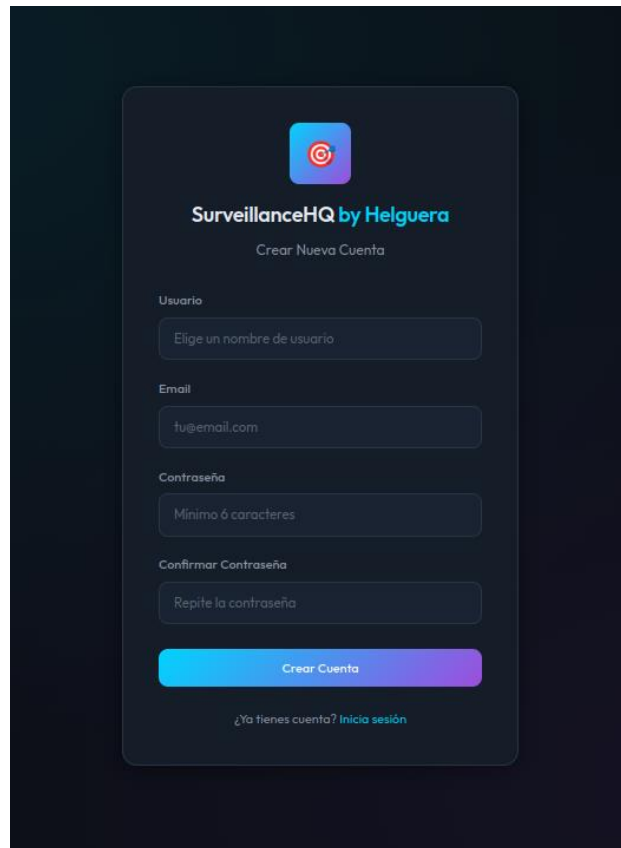
The image shows a user registration form titled "SurveillanceHQ by Helguera" with the subtitle "Crear Nueva Cuenta". The form is set against a dark background. It includes input fields for "Usuario" (with placeholder "Elige un nombre de usuario"), "Email" (with placeholder "tu@email.com"), "Contraseña" (with placeholder "Mínimo 6 caracteres"), and "Confirmar Contraseña" (with placeholder "Repite la contraseña"). A prominent blue and purple gradient button labeled "Crear Cuenta" is at the bottom. Below the button is a link that says "¿Ya tienes cuenta? Inicia sesión".

Ilustración 16. Ventana de registro de usuario

5.5.6.3. Dashboard principal

El dashboard principal (Ilustración 17) constituye la pantalla inicial del sistema y se muestra automáticamente una vez que el usuario ha iniciado sesión de forma correcta. Esta vista centraliza la información más relevante del estado del sistema y proporciona accesos directos a las principales funcionalidades de la aplicación.

En el lateral izquierdo se dispone un menú de navegación con tres accesos principales. El primero permite regresar en cualquier momento a la pantalla principal del dashboard. El segundo da acceso a la sección de gestión de cámaras, desde donde el usuario puede administrar los dispositivos registrados en el sistema. El tercer acceso conduce al apartado de gestión de alertas, en el que se muestran los eventos detectados y su estado. En la parte inferior del menú se indica el nombre del usuario autenticado y se incluye un botón que permite cerrar la sesión de forma explícita.

La zona central del dashboard se organiza de arriba hacia abajo. En la parte superior se presentan las estadísticas generales del sistema mediante cuatro bloques informativos. Estos indicadores muestran, respectivamente, el número total de cámaras registradas, el número de cámaras activas en ese momento, el total de alertas generadas durante el día actual y el número de alertas que aún no han sido consultadas por el usuario. Esta información proporciona una visión rápida y resumida del estado global del sistema.

Debajo de estos indicadores se muestra la visualización en tiempo real de las cámaras activas. En esta sección se representan los flujos de vídeo en directo y, cuando se detecta una persona, se superponen las correspondientes “bounding boxes”, facilitando la interpretación visual de las detecciones realizadas por el sistema.

Finalmente, en la parte inferior del dashboard se presenta un listado con las alertas más recientes, mostrando por defecto las cinco últimas. Para acceder al historial completo de alertas, el usuario puede utilizar el apartado específico de alertas disponible en el menú lateral o el botón “Ver todas” incluido en esta sección.

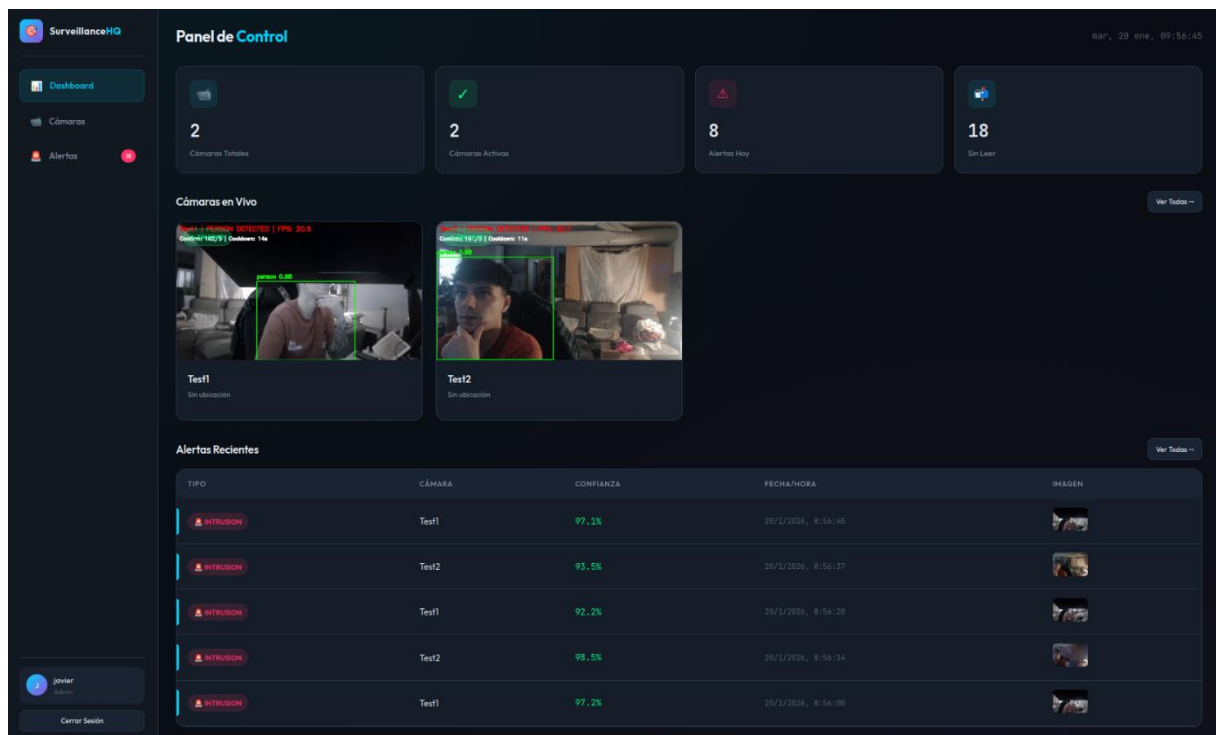


Ilustración 17. Ventana principal del dashboard

5.5.6.4. Gestion de cámaras

Desde el apartado de gestión de cámaras es posible supervisar en tiempo real el flujo de vídeo recibido y administrar los dispositivos registrados en el sistema. Esta vista permite al usuario controlar de forma centralizada el funcionamiento de cada cámara y ajustar su configuración según las necesidades del entorno monitorizado.

Tal y como se muestra en la Ilustración 18, cada cámara dispone de tres acciones principales. La primera permite iniciar o detener el proceso de detección asociado a la cámara. Como se ha descrito anteriormente, cada dispositivo puede encontrarse en estado activo o desactivado, lo que posibilita mantener cámaras registradas en el sistema sin que participen en el proceso de vigilancia en un momento determinado, evitando así su eliminación.

Adicionalmente, se incluyen dos acciones orientadas a la gestión de la configuración de cada cámara. Una de ellas permite modificar los parámetros asociados al dispositivo, mientras que la otra posibilita su eliminación completa del sistema. En este último caso, la supresión de una cámara conlleva también la eliminación de las alertas asociadas a la misma, garantizando la coherencia de la información almacenada

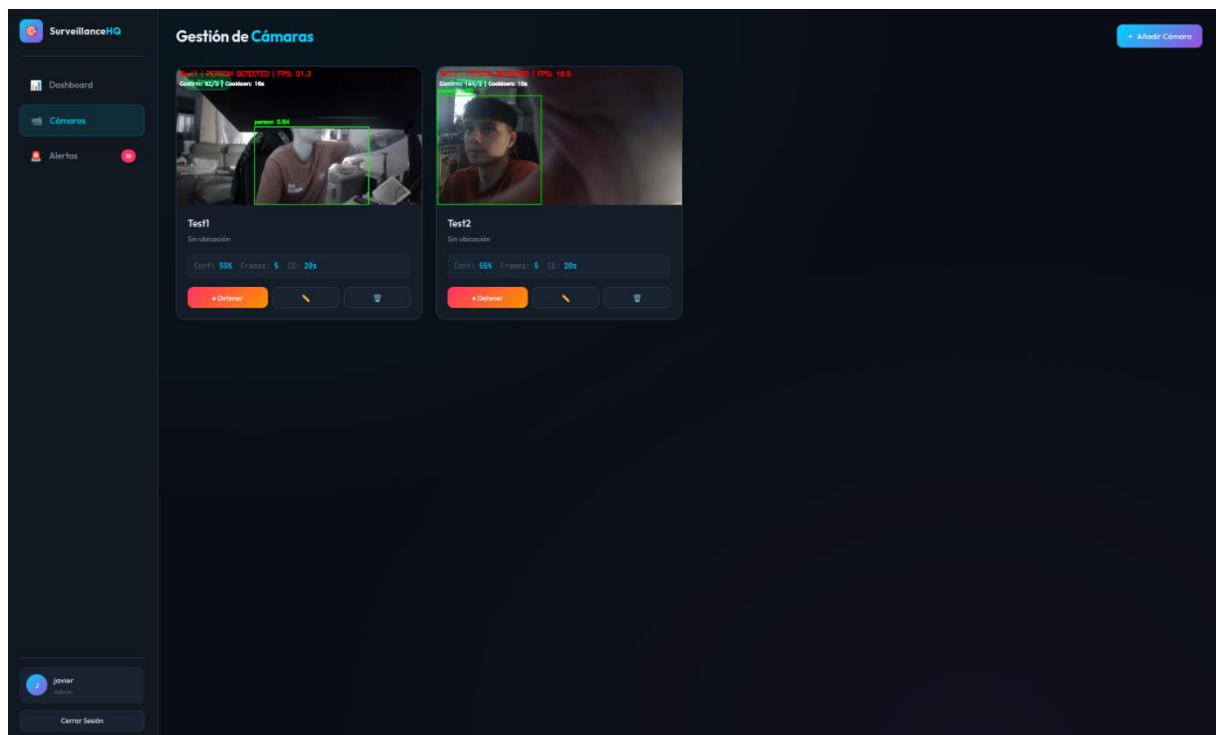


Ilustración 18. Ventana de gestión de cámaras del sistema

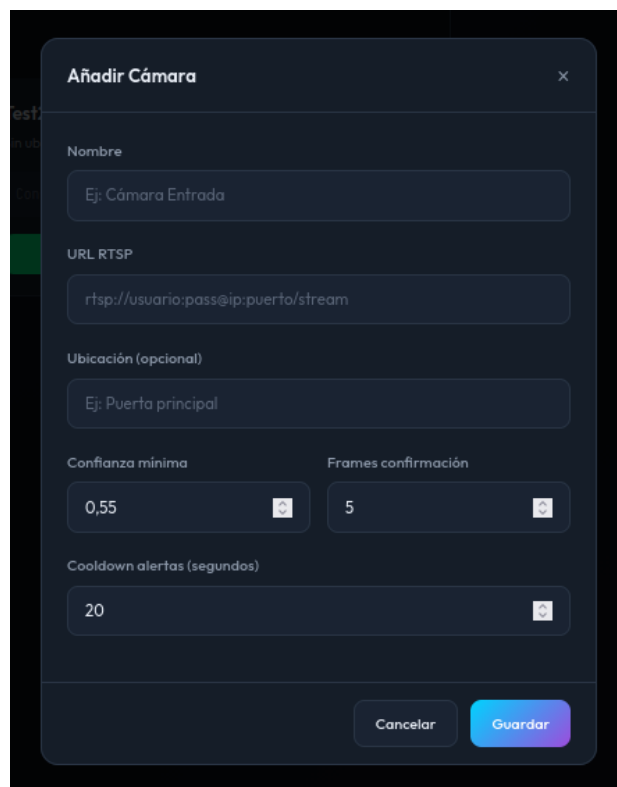
Desde esta misma página, en la esquina superior derecha, se encuentra un botón que permite añadir nuevas cámaras al sistema. Al pulsarlo, se abre una ventana modal como la que se

muestra en la Ilustración 19, desde la cual el usuario puede registrar un nuevo dispositivo de captura de forma guiada.

El formulario de alta de cámaras permite introducir un nombre identificativo para la cámara, que facilita su reconocimiento posterior dentro del dashboard, así como la URL RTSP correspondiente al flujo de vídeo. Adicionalmente, se incluye un campo opcional de ubicación, pensado para aportar contexto al usuario sobre la posición física de la cámara dentro del entorno monitorizado, como una habitación o zona concreta de la vivienda.

La ventana incorpora también parámetros básicos de configuración del proceso de detección asociados a la cámara. Entre ellos se encuentran el umbral mínimo de confianza, el número de frames necesarios para confirmar una detección y el tiempo de cooldown entre alertas consecutivas. Estos valores permiten ajustar el comportamiento del sistema en función de las características de cada cámara y del entorno en el que se encuentra instalada.

Finalmente, el formulario incluye opciones para cancelar la operación o guardar la nueva cámara. Al confirmar el registro, la cámara queda almacenada en el sistema y pasa a estar disponible en el apartado de gestión de cámaras, desde donde puede activarse y comenzar el proceso de detección cuando el usuario lo considere oportuno.



Añadir Cámara ✕

Nombre
Ej: Cámara Entrada

URL RTSP
rtsp://usuario:pass@ip:puerto/stream

Ubicación (opcional)
Ej: Puerta principal

Confianza mínima: 0,55

Frames confirmación: 5

Cooldown alertas (segundos): 20

Cancelar Guardar

Ilustración 19. Ventana para añadir una cámara al sistema

5.5.6.5. Gestión de alertas

La ventana de gestión de alertas (Ilustración 20) permite consultar todas las alertas generadas por el sistema, presentadas de forma ordenada cronológicamente. Esta vista ofrece al usuario una visión completa del historial de eventos detectados y facilita su revisión y gestión.

En la parte superior derecha se incluyen distintos controles de filtrado que permiten seleccionar las alertas en función de la cámara que las ha generado y de su estado, distinguiendo entre alertas leídas y no leídas. Asimismo, se ha incorporado un botón que permite marcar todas las alertas como leídas de forma simultánea, simplificando la gestión cuando se acumulan múltiples eventos pendientes.

Cada alerta muestra información relevante sobre el evento detectado, incluyendo la cámara asociada, el nivel de confianza de la detección, la fecha y hora en la que se produjo y una captura de imagen que actúa como evidencia visual. Para cada alerta se proporcionan dos acciones: una para marcarla individualmente como leída y otra para eliminarla del sistema.

Este diseño permite al usuario revisar y organizar las alertas de manera eficiente, manteniendo un control claro sobre los eventos detectados por el sistema.

TIPO	CÁMARA	DETALLE	CONFIANZA	FECHA/HORA	IMAGEN	ACCIONES
INTRUSION	Test1	Person detected conf=0.97 confirm_frames=415	97.1%	20 ene 08:54:49		☒
INTRUSION	Test2	Person detected conf=0.94 confirm_frames=5	93.5%	20 ene 08:54:57		☒
INTRUSION	Test1	Person detected conf=0.92 confirm_frames=16	92.2%	20 ene 08:54:59		☒
INTRUSION	Test2	Person detected conf=0.99 confirm_frames=5	98.5%	20 ene 08:55:14		☒
INTRUSION	Test1	Person detected conf=0.97 confirm_frames=601	97.2%	20 ene 08:56:00		☒
INTRUSION	Test1	Person detected conf=0.99 confirm_frames=600	98.1%	20 ene 08:55:40		☒
INTRUSION	Test1	Person detected conf=0.99 confirm_frames=600	98.6%	20 ene 08:55:29		☒
INTRUSION	Test1	Person detected conf=0.98 confirm_frames=5	97.9%	20 ene 08:55:09		☒
INTRUSION	Test2	Person detected conf=0.95 confirm_frames=5	95.4%	18 ene 14:51:06		☒
INTRUSION	Test1	Person detected conf=0.77 confirm_frames=5	77.4%	18 ene 14:51:04		☒
INTRUSION	Test1	Person detected conf=0.94 confirm_frames=390	94.1%	18 ene 14:50:39		☒
INTRUSION	Test2	Person detected conf=0.77 confirm_frames=266	76.8%	18 ene 14:50:12		☒
INTRUSION	Test1	Person detected conf=0.99 confirm_frames=468	98.7%	18 ene 14:50:35		☒
INTRUSION	Test2	Person detected conf=0.72 confirm_frames=5	72.3%	18 ene 14:50:02		☒

Ilustración 20. Ventana de gestión de alertas del sistema

5.6. BOT DE TELEGRAM CON ACCESO LIMITADO DESDE EL EXTERIOR

Un sistema de vigilancia pierde gran parte de su utilidad si no es capaz de notificar eventos relevantes cuando el usuario se encuentra fuera de la red local. Por este motivo, se ha desarrollado un módulo específico que permite la integración con la aplicación de mensajería Telegram, proporcionando un canal externo para el envío de alertas al usuario independientemente de su ubicación.

Dado que uno de los pilares fundamentales del proyecto es la privacidad y el procesamiento completamente local, el uso de Telegram se ha restringido deliberadamente a funciones mínimas y controladas. Su finalidad se limita exclusivamente al envío de notificaciones ante eventos detectados y a la posibilidad de consultar el estado general de las cámaras. En ningún caso se expone el flujo de vídeo ni se permite el acceso remoto directo a elementos internos de la red doméstica, preservando así el aislamiento del sistema y evitando riesgos de seguridad adicionales.

Para implementar esta funcionalidad se ha desarrollado un servicio independiente denominado `telegram_bot.py`. Esta solución se beneficia del enfoque modular adoptado en el diseño del sistema, que permite incorporar nuevos módulos de manera desacoplada y fácilmente integrable. Aunque en este caso se aplica a un bot de mensajería, la misma estructura podría extenderse en el futuro a otros componentes complementarios de seguridad, como sensores de apertura de puertas, detectores de movimiento adicionales o sistemas domóticos integrados.

5.6.1. Creación del bot

La solución adopta una separación clara de responsabilidades mediante dos clases: `TelegramNotifier`, encargada de la interacción directa con la API de Telegram y de la ejecución del polling, y `TelegramManager`, que actúa como punto de acceso único (patrón Singleton) para el resto de la aplicación. Esta separación permite que la lógica de negocio (por ejemplo, la detección de intrusiones) invoque un método síncrono de alto nivel sin depender de detalles de concurrencia o de `asyncio`.

El núcleo funcional reside en el método asíncrono `send_alert(...)`, perteneciente a `TelegramNotifier`, responsable de construir y enviar una notificación. El mensaje incluye información contextual como el nombre de la cámara, el tipo de evento y un valor de

confianza asociado a la detección. Además, se incorpora una marca temporal para registrar el instante exacto en el que se genera la alerta. La construcción del texto se realiza mediante interpolación controlada y se estructura de forma legible para el receptor. Un fragmento representativo de esta composición es el siguiente:

```
timestamp = datetime.now().strftime("%d/%m/%Y %H:%M:%S")

message = (
    "🚨 ALERTA DE INTRUSIÓN\n\n"
    f"📷 Cámara: {camera_name}\n"
    f"⚠️ Tipo: {event_type}\n"
    f"📊 Confianza: {confidence:.1%}\n"
    f"🕒 Hora: {timestamp}\n"
)
```

En el escenario considerado, se asume que la evidencia visual del evento está siempre disponible. Por tanto, el envío se realiza sistemáticamente mediante el método `send_photo`, que permite adjuntar el fichero al mensaje como caption. El bloque de envío es el siguiente:

```
with open(image_path, "rb") as photo:
    await self.bot.send_photo(
        chat_id=self.chat_id,
        photo=photo,
        caption=message
    )
```

El uso de programación asíncrona resulta esencial porque la interacción con la API de Telegram implica operaciones de entrada/salida en red, que no deben bloquear el sistema principal. Para resolver esta convivencia, se implementa un método envoltorio `send_alert_sync(...)` que permite invocar el envío desde un contexto no asíncrono sin reestructurar el resto de la aplicación. El enfoque adoptado consiste en programar la corrutina dentro del bucle de eventos activo del bot mediante `asyncio.run_coroutine_threadsafe`, lo cual constituye un mecanismo seguro de comunicación entre hilos:

```
fut = asyncio.run_coroutine_threadsafe(
    self.send_alert(...),
    self._loop
)
return fut.result(timeout=10)
```

Otro aspecto central del módulo es la ejecución del bot de Telegram en segundo plano. Para no interferir con el proceso principal, el sistema lanza el polling en un hilo dedicado. Esta

elección permite que el bot permanezca escuchando comandos y eventos sin bloquear la ejecución de otros subsistemas, como la captura de vídeo o el análisis mediante inteligencia artificial. El arranque se realiza mediante:

```
self._thread = threading.Thread(  
    target=self._run_polling,  
    daemon=True  
)  
self._thread.start()
```

Dentro del hilo, se crea un bucle de eventos independiente asociado exclusivamente a ese contexto de ejecución. Esto es relevante porque asyncio gestiona sus event loops por hilo, y la separación evita conflictos con otros bucles que pudieran existir en el programa. En este loop se inicializa el objeto Application de python-telegram-bot, se registran los manejadores de comandos y se inicia el proceso de polling:

```
self._application = Application.builder().token(self.token).build()  
self._application.add_handler(CommandHandler("start", self._cmd_start))  
self._application.add_handler(CommandHandler("status", self._cmd_status))  
  
await self._application.initialize()  
await self._application.start()  
await self._application.updater.start_polling()
```

Los comandos implementados (/start, /help, /status) permiten una interacción mínima pero funcional con el sistema. En particular, el comando /status actúa como mecanismo de observabilidad, ya que permite consultar remotamente el estado actual de las cámaras. Para evitar un acoplamiento fuerte entre el bot y el subsistema de cámaras, se emplea un patrón de inyección mediante callback. El bot no conoce internamente cómo se calculan los estados, sino que invoca una función externa proporcionada por el sistema:

```
statuses = self._get_cameras_status()
```

Posteriormente, se genera un resumen textual para el usuario con indicadores visuales de actividad (por ejemplo, cámara activa o detenida). Este diseño facilita la extensión futura, ya que el bot podría consultar cualquier otro subsistema simplemente incorporando nuevos callbacks o comandos.

La clase TelegramManager completa el diseño proporcionando un punto de acceso único al servicio de notificaciones. Su implementación como Singleton garantiza que el bot no se instancie múltiples veces, evitando errores típicos como la duplicación de procesos de polling o el uso simultáneo del mismo token en varias instancias. Desde la perspectiva del resto de la aplicación, el envío de una alerta se reduce a una llamada directa:

```
telegram_manager.send_alert(  
    camera_name="Entrada",  
    event_type="Intrusión",  
    confidence=0.92,  
    image_path="/tmp/event.jpg"  
)
```

5.6.2. Configuración y puesta en marcha

Es importante destacar que el bot de Telegram no corresponde a un bot genérico compartido entre múltiples usuarios encargado de distribuir alertas de manera global. Por el contrario, cada usuario debe crear y configurar su propio bot, de forma que este únicamente funcionará de manera individual y recibirá exclusivamente las notificaciones asociadas a dicho usuario. Con el fin de facilitar este proceso, se ha incorporado una sección específica de configuración, accesible desde el menú lateral izquierdo del panel de control (Ilustración 21).

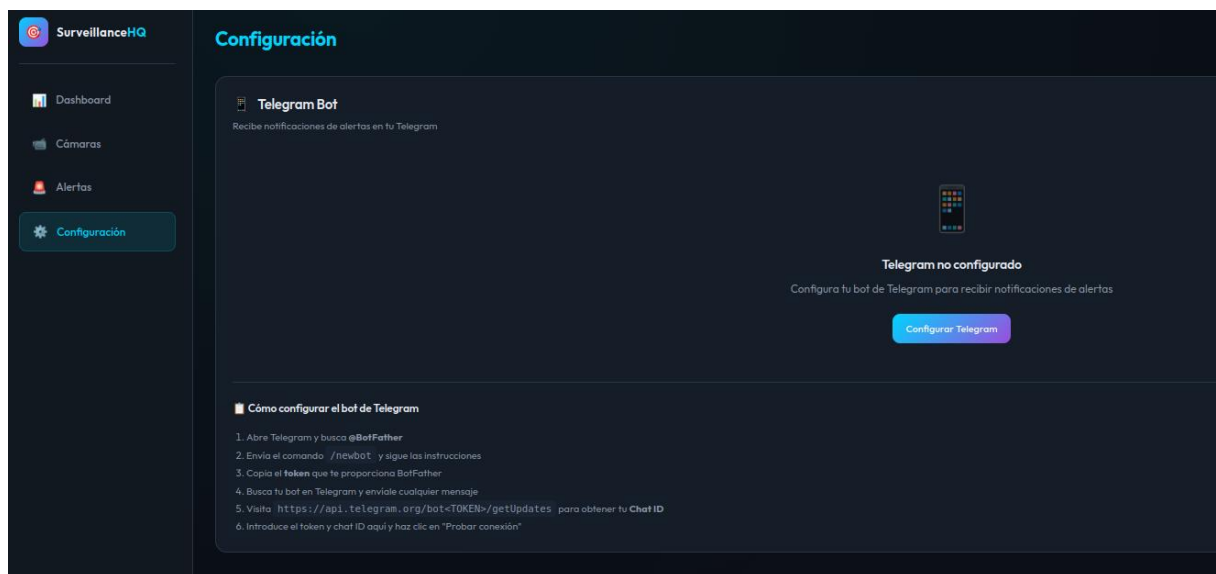


Ilustración 21. Ventana de configuración del bot de Telegram

Las instrucciones necesarias para el proceso de configuración se presentan directamente al usuario en el dashboard (Ilustración 21). En primer lugar, es necesario iniciar una conversación con el bot oficial de Telegram, @BotFather.

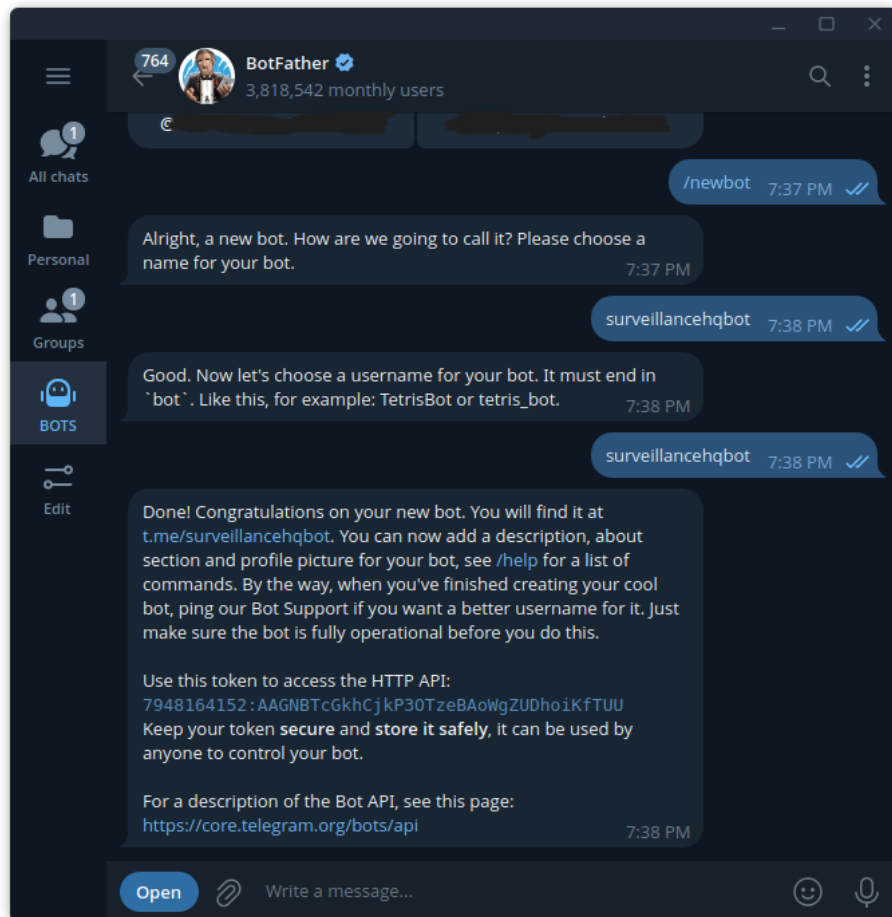


Ilustración 22. Conversación con BotFather en Telegram para la creación del bot

La conversación completa se puede ver en la Ilustración 22. Una vez establecido el nombre de nuestro bot, BotFather nos devolverá un token único. Es muy importante no compartir dicho token con nadie, ya que tendrá acceso completo al bot.

El segundo paso es encontrar el id único de nuestro usuario, es importante para que el bot sepa a quién tiene que enviar los mensajes con las alertas. Para ello, se accede a la conversación con el bot que se ha creado y se envía un mensaje, cualquier cosa valdrá (Ilustración 23).

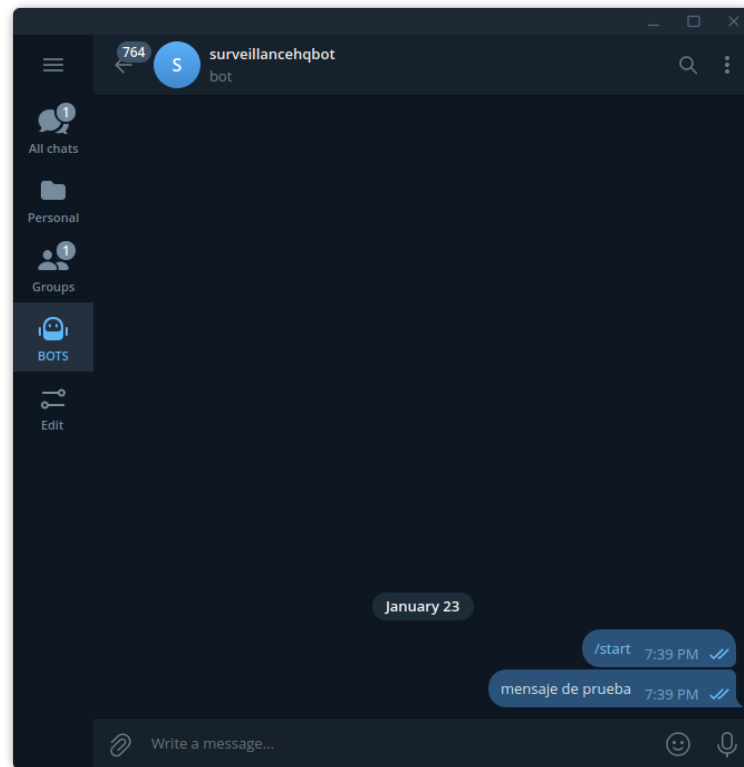


Ilustración 23. Envío del primer mensaje al nuevo bot

Si se accede a la url:

https://api.telegram.org/bot<YOUR_TOKEN>/getUpdates

Reemplazando “YOUR_TOKEN” con el token que BotFather ha enviado, se podrá ver información acerca de los mensajes que ha recibido, junto con el identificador único de quien los envió:



Ilustración 24. Información disponible en la URL getUpdates

El campo que se necesita es “id”. Ahora que ya se cuenta tanto con el token del bot como con el id del usuario, se puede configurar el bot desde el dashboard (Ilustración 25).

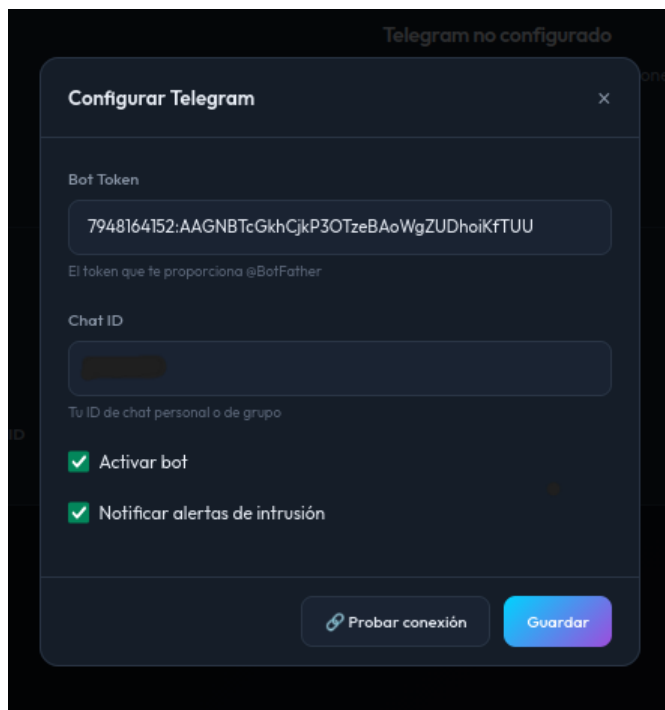


Ilustración 25. Ventana de configuración del bot desde el dashboard

Una vez completada la configuración, el bot queda preparado y operativo. A partir de ese momento, cada vez que el sistema detecte una intrusión, además de generarse la alerta correspondiente dentro de la plataforma, se enviará automáticamente una notificación a través del bot. Este mensaje incluirá información relevante como la cámara implicada, el nivel de confianza de la detección, la hora del suceso y, de manera destacada, la imagen asociada a la alerta (Ilustración 26).

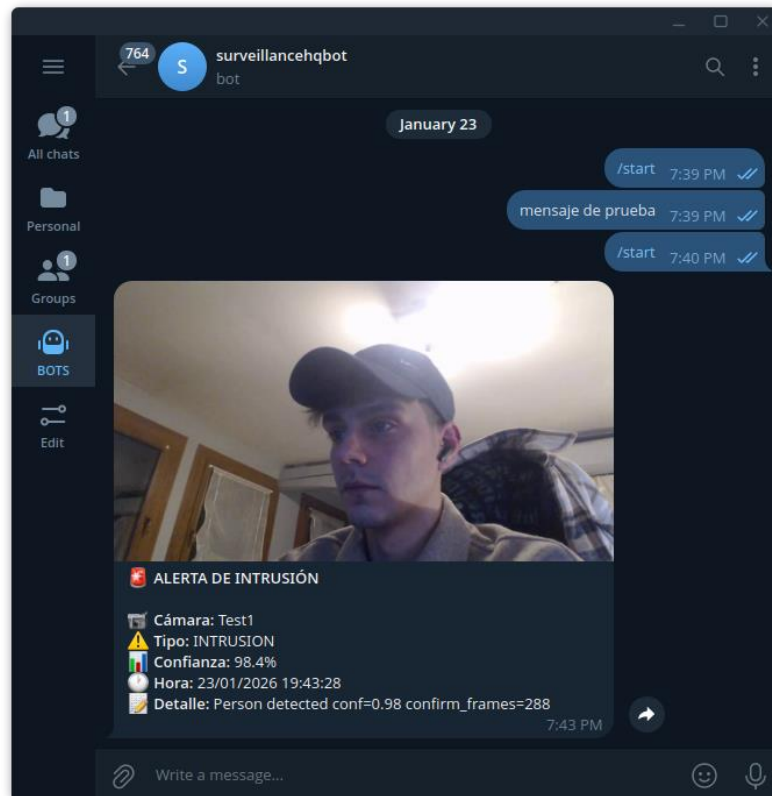


Ilustración 26. Ejemplo de una alerta de intrusión recibido en el bot

Como funcionalidad adicional, el bot permite al usuario enviar el comando “/status”, mediante el cual se obtiene un resumen del estado actual de las cámaras del sistema. Esta opción resulta especialmente útil para comprobar de forma rápida si el sistema se encuentra operando correctamente (Ilustración 27).

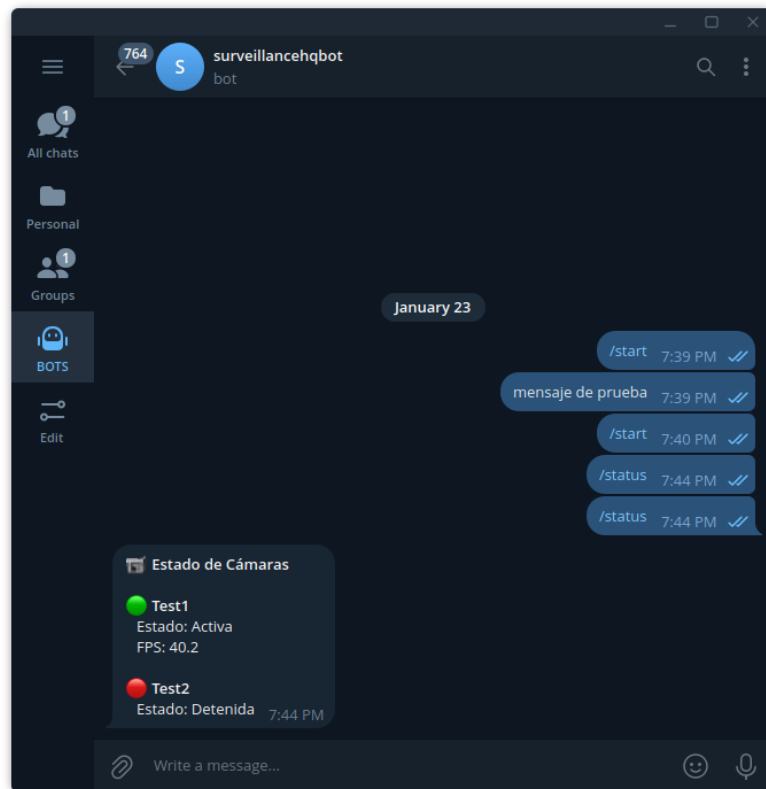


Ilustración 27. Ejemplo de respuesta del bot al comando /status

5.7.CONTENEDOR DOCKER LISTO PARA EL DESPLIEGUE

La utilización de contenedores Docker para desplegar el sistema aporta numerosas ventajas desde el punto de vista del desarrollo, la distribución y la operación. En primer lugar, permite encapsular en un único entorno aislado todos los componentes necesarios (dependencias, librerías y configuraciones) garantizando que la aplicación se ejecute de forma consistente independientemente del sistema operativo o la máquina donde se despliegue. Esto reduce significativamente los problemas derivados de diferencias entre entornos de desarrollo y producción.

Además, Docker facilita la portabilidad y la reproducibilidad del sistema, ya que el contenedor puede trasladarse e iniciarse fácilmente en cualquier infraestructura compatible, simplificando el proceso de instalación para el usuario final. Asimismo, el aislamiento proporcionado por los contenedores mejora la seguridad y la estabilidad, evitando conflictos con otras aplicaciones instaladas en el host. Finalmente, este enfoque también contribuye a una gestión más eficiente del ciclo de vida del sistema, ya que actualizaciones, escalado o despliegues pueden realizarse de manera más controlada y automatizada.

5.7.1. Dockerfile

Se ha creado un fichero Dockerfile en el proyecto. Este fichero, cuyo código completo puede encontrarse en el Anexo B, describe el proceso de construcción de la imagen de forma estructurada, empleando una estrategia de multi-stage build que permite reducir el tamaño final del contenedor y separar claramente las dependencias de compilación de las necesarias en ejecución. En una primera etapa, basada en la imagen `python:3.13-slim`, se instala únicamente el conjunto mínimo de herramientas de compilación requeridas para construir dependencias de Python que lo necesiten. A continuación, se crea un entorno virtual aislado dentro del contenedor (`/opt/venv`) y se instalan en él todos los paquetes especificados en el fichero `requirements.txt`.

En la segunda etapa, destinada ya al entorno de ejecución final, se vuelve a partir de una imagen `python:3.13-slim`, pero esta vez instalando únicamente las librerías del sistema necesarias para el correcto funcionamiento de la aplicación, incluyendo dependencias multimedia como `ffmpeg` y componentes de soporte para procesamiento de vídeo. Posteriormente, se copia el entorno virtual generado en la etapa anterior, evitando así tener que instalar de nuevo todas las dependencias, lo que contribuye a mejorar la eficiencia y limpieza de la imagen resultante.

El contenedor configura como directorio de trabajo `/app` y copia tanto el código fuente como los modelos utilizados por el sistema. Además, se crean directorios internos destinados al almacenamiento persistente de datos y evidencias generadas durante la ejecución. También se definen variables de entorno que facilitan la configuración del sistema de manera flexible y desacoplada del código, indicando rutas internas para datos, evidencias y modelos.

Finalmente, el contenedor expone el puerto 8000, correspondiente al servicio web principal, e incluye un `healthcheck` que permite verificar periódicamente el estado del servidor mediante una petición HTTP a un endpoint interno. La ejecución se realiza mediante `uvicorn`, que actúa como servidor ASGI para desplegar la aplicación desarrollada con FastAPI.

5.7.2. Docker Compose

Docker Compose es una herramienta complementaria a Docker que permite definir y gestionar aplicaciones formadas por uno o varios contenedores mediante un único fichero declarativo (`docker-compose.yml`). Su principal utilidad radica en que simplifica el despliegue

del sistema, ya que permite configurar de manera centralizada aspectos como la construcción de imágenes, la exposición de puertos, la persistencia de datos o las variables de entorno, evitando la necesidad de ejecutar manualmente múltiples comandos docker run. De este modo, Docker Compose resulta especialmente útil para proyectos complejos o aplicaciones que requieren una configuración reproducible y fácil de desplegar.

En este proyecto, el fichero Compose (incluido completo en el anexo) define el servicio principal denominado surveillance, encargado de ejecutar el sistema completo dentro de un contenedor. En primer lugar, se especifica que la imagen debe construirse a partir del Dockerfile del propio proyecto, indicando como contexto el directorio raíz. Esto permite que el usuario pueda levantar el sistema simplemente ejecutando docker-compose up, sin necesidad de construir manualmente la imagen.

El servicio asigna un nombre al contenedor (surveillance-hq) y expone el puerto 8000 del contenedor hacia el host mediante el mapeo "8000:8000", lo que posibilita acceder a la interfaz web del sistema desde el navegador. Asimismo, se configuran volúmenes persistentes para almacenar información relevante generada durante la ejecución. En particular, se mantienen separados los datos internos del sistema (/app/data) y las evidencias capturadas durante las alertas (/app/evidences). Gracias a esta estrategia, dichos ficheros permanecen disponibles incluso si el contenedor se elimina o se reinicia.

Adicionalmente, Compose permite definir variables de entorno necesarias para el funcionamiento del sistema. En este caso, se incluye una clave secreta (SECRET_KEY) configurable externamente y la zona horaria del sistema, garantizando que los registros y marcas temporales se ajusten al contexto del despliegue. También se establece una política de reinicio automático (restart: unless-stopped), lo que incrementa la robustez del sistema al permitir que el contenedor se recupere automáticamente ante fallos inesperados.

El archivo incluye un mecanismo de healthcheck equivalente al definido en el Dockerfile, mediante el cual se realizan comprobaciones periódicas al endpoint interno /health. Esto permite a Docker supervisar el estado del servicio y detectar posibles situaciones en las que la aplicación no esté funcionando correctamente.

Con los dos ficheros en el mismo directorio que el resto del proyecto, con un simple comando se lanzará toda la aplicación completa automáticamente:

```
docker compose up -d
```

Y se podrá acceder desde la red local en el puerto 8000:

```
https://localhost:8000
```

5.8. GUÍA DE USUARIO

El sistema desarrollado ha sido diseñado para facilitar su despliegue y uso por parte de un usuario doméstico sin necesidad de configuraciones complejas. Para ello, toda la aplicación se distribuye mediante contenedores Docker, permitiendo una instalación rápida y reproducible en cualquier servidor local compatible, como una Raspberry Pi o un mini ordenador con Linux.

El código completo se puede consultar en el repositorio GitHub del proyecto:
<https://github.com/Helguera/TFM-AI-UNIR-2026>

La puesta en marcha del sistema se realiza ejecutando el archivo `docker-compose.yml`, que lanza automáticamente todos los servicios necesarios (backend, base de datos y módulos de detección) mediante el comando estándar `docker compose up`. Una vez iniciado, el usuario puede acceder al dashboard web desde el navegador para gestionar el sistema de forma centralizada, previa creación de un usuario.

Desde esta interfaz es posible registrar cámaras IP proporcionando su URL RTSP y ajustar parámetros básicos de funcionamiento. El sistema comenzará a analizar el vídeo en tiempo real y generará eventos cuando se detecte presencia humana, almacenando las alertas asociadas para su consulta posterior.

El usuario puede configurar el módulo opcional de Telegram desde el propio panel web, introduciendo el token del bot y el identificador de chat. De este modo, el sistema puede enviar notificaciones inmediatas ante detecciones relevantes, manteniendo siempre el procesamiento local y limitando la exposición externa únicamente al canal de alertas.

6. EVALUACIÓN

Con el objetivo de evaluar el rendimiento del sistema desarrollado, se ha llevado a cabo una fase de pruebas orientada a medir tanto la eficacia del proceso de detección como su comportamiento en condiciones reales de funcionamiento. Esta etapa resulta fundamental para determinar el grado de fiabilidad alcanzado por la solución propuesta y comprobar su aplicabilidad en entornos domésticos o de vigilancia con recursos limitados.

El sistema ha sido instalado y ejecutado en una Raspberry Pi 5, actuando como plataforma principal de procesamiento. Para la captura de vídeo en tiempo real, se ha utilizado una cámara de seguridad Tapo, conectada mediante el protocolo RTSP, lo que permite simular un escenario real de monitorización continua. Esta configuración refleja un caso de uso representativo en sistemas embebidos de bajo coste, donde es esencial garantizar una ejecución eficiente sin comprometer la precisión del modelo.

Las pruebas realizadas consisten en diferentes situaciones controladas en las que una persona pasa por delante del dispositivo en diversas condiciones. De este modo, se busca analizar la robustez del sistema frente a factores externos que pueden influir directamente en la calidad de la imagen y, por tanto, en el rendimiento del algoritmo.

A partir de estos ensayos, se recopilan resultados que permiten medir aspectos clave como la tasa de detección, la estabilidad del seguimiento y la capacidad de respuesta en tiempo real, ofreciendo una visión global sobre la eficacia obtenida en el despliegue práctico del sistema.

6.1.METODOLOGÍA EMPÍRICA

Para la medición del rendimiento se han diseñado una serie de pruebas empíricas controladas basadas en situaciones reales de detección. Estas pruebas consisten en el paso de un sujeto por delante del campo de visión de la cámara bajo diferentes condiciones:

1. Movimiento frontal hacia la cámara.
2. Movimiento de espaldas.
3. Presencia parcial del cuerpo.
4. Luz diurna.
5. Luz nocturna.

Cada uno de los escenarios descritos se repitió un total de 10 veces, evaluándose además el comportamiento del sistema a diferentes distancias respecto al dispositivo de captura: 1 metro, 5 metros y 10 metros.

Adicionalmente, con el fin de estudiar el impacto del umbral de detección en la precisión del sistema, todas las pruebas se realizaron empleando distintos niveles de confianza del modelo, establecidos en 25%, 50% y 75%.

6.2.MÉTRICAS EMPLEADAS

Métrica	Descripción	Fórmula
TP (Verdaderos Positivos)	Número de casos en los que el sistema detecta correctamente la presencia del sujeto.	
FP (Falsos Positivos)	Número de detecciones erróneas generadas cuando no existe realmente un objetivo válido.	
FN (Falsos Negativos)	Número de ocasiones en las que el sujeto aparece en escena, pero el sistema no lo detecta.	
Precisión	Proporción de detecciones correctas respecto al total de detecciones realizadas.	$Precision = \frac{TP}{TP + FP}$
Recall	Mide la capacidad del sistema para detectar correctamente los objetivos reales presentes en la escena.	$Recall = \frac{TP}{TP + FN}$

F1-Score	Medida combinada que proporciona un equilibrio entre precisión y sensibilidad.	$F1 = \frac{Precision \times Recall}{Precision + Recall}$
-----------------	--	---

6.3.RESULTADOS DE EFICACIA DE DETECCIÓN

Los resultados experimentales recogidos en la validación (Tabla 3) permiten evaluar de forma objetiva el rendimiento del sistema bajo distintas configuraciones de umbral de confianza y a diferentes distancias respecto a la cámara (1 m, 5 m y 10 m). En términos generales, el sistema muestra un comportamiento consistente, con valores medios de precisión cercanos al 0.87, lo que confirma la viabilidad del enfoque para escenarios domésticos. En el escenario de movimiento frontal hacia la cámara, los mejores resultados se obtienen a corta distancia. Con un umbral del 25%, se alcanza una precisión de 0.91 a 1 metro (10 verdaderos positivos y solo 1 falso positivo), manteniéndose en torno a 0.90 a 5 metros. Sin embargo, a 10 metros se observa una reducción más notable (precisión de 0.80), aumentando los falsos negativos debido a la menor presencia visual del individuo en el fotograma.

En configuraciones más restrictivas, como el umbral del 75%, se observa una clara reducción de falsos positivos. Por ejemplo, a 1 metro se consigue una precisión perfecta de 1.00, al no registrarse falsas alarmas. No obstante, esta mejora conlleva una ligera pérdida de sensibilidad, ya que a mayores distancias aparecen más falsos negativos: a 10 metros la precisión se sitúa en 0.88, pero con un número mayor de detecciones perdidas (FN = 3).

De forma global, los resultados muestran el compromiso esperado entre sensibilidad y precisión:

- Umbrales bajos (25%) favorecen la detección, pero incrementan ligeramente las falsas alarmas.
- Umbrales altos (75%) minimizan falsos positivos, aunque pueden reducir la capacidad de detección en escenas menos favorables.

La distancia es también un factor determinante: el sistema mantiene un alto rendimiento hasta aproximadamente 5 metros, mientras que a 10 metros la precisión tiende a disminuir debido a la reducción de tamaño del objeto detectado. Estos resultados validan que el sistema

logra un equilibrio adecuado para videovigilancia doméstica local, destacando especialmente configuraciones intermedias (50%) como opción óptima entre fiabilidad y reducción de falsas alarmas.

Tabla 3. Métricas obtenidas en las pruebas de detección (TP, FP, FN y precisión) en función de la distancia (1 m, 5 m, 10 m) y el umbral de confianza configurado.

		25% Confianza			50% Confianza			75% Confianza		
		1M	5M	10M	1M	5M	10M	1M	5M	10M
Movimiento frontal hacia la cámara	TP	10	9	8	10	9	8	9	8	7
	FP	1	1	2	1	1	2	0	1	1
	FN	0	1	2	0	1	2	1	2	3
	Precision	0.91	0.90	0.80	0.91	0.90	0.80	1.00	0.89	0.88
	Recall	1.00	0.90	0.80	1.00	0.90	0.80	0.90	0.80	0.70
	F1-Score	0.48	0.45	0.40	0.48	0.45	0.40	0.47	0.42	0.39
Movimiento de espaldas	TP	9	8	7	9	8	6	8	7	5
	FP	2	3	3	1	2	2	0	1	1
	FN	1	2	3	1	2	4	2	3	5
	Precision	0.82	0.73	0.70	0.90	0.80	0.75	1.00	0.88	0.83
	Recall	0.90	0.80	0.70	0.90	0.80	0.60	0.80	0.70	0.50
	F1-Score	0.43	0.38	0.35	0.45	0.40	0.33	0.44	0.39	0.31
Presencia parcial del cuerpo	TP	8	7	5	8	6	4	6	5	3
	FP	3	3	4	2	2	3	1	1	2
	FN	2	3	5	2	4	6	4	5	7
	Precision	0.73	0.70	0.56	0.80	0.75	0.57	0.86	0.83	0.60
	Recall	0.80	0.70	0.50	0.80	0.60	0.40	0.60	0.50	0.30
	F1-Score	0.38	0.35	0.26	0.40	0.33	0.24	0.35	0.31	0.20
Luz diurna	TP	10	9	9	10	9	8	9	8	7
	FP	2	2	3	1	1	2	0	1	1
	FN	0	1	1	0	1	2	1	2	3
	Precision	0.83	0.82	0.75	0.91	0.90	0.80	1.00	0.89	0.88
	Recall	1.00	0.90	0.90	1.00	0.90	0.80	0.90	0.80	0.70
	F1-Score	0.45	0.43	0.41	0.48	0.45	0.40	0.47	0.42	0.39
Luz nocturna	TP	8	6	4	7	5	3	6	4	2
	FP	3	3	4	2	2	3	1	1	2
	FN	2	4	6	3	5	7	4	6	8
	Precision	0.73	0.67	0.50	0.78	0.71	0.50	0.86	0.80	0.50
	Recall	0.80	0.60	0.40	0.70	0.50	0.30	0.60	0.40	0.20
	F1-Score	0.38	0.32	0.22	0.37	0.29	0.19	0.35	0.27	0.14

Los valores medios recogidos en estas tablas (Tabla 4, Tabla 5, Tabla 6, Tabla 7 y Tabla 8) permiten observar de forma global el comportamiento del sistema bajo las distintas configuraciones evaluadas. En general, se aprecia que la precisión se mantiene elevada a distancias cortas y medias, mientras que a mayores distancias el rendimiento tiende a disminuir ligeramente debido a la menor presencia visual del individuo en la escena.

Asimismo, los resultados muestran el compromiso entre sensibilidad y fiabilidad: umbrales de confianza más bajos favorecen la detección, mientras que valores más altos reducen la aparición de falsas alarmas. Estas medias permiten identificar configuraciones intermedias como las más equilibradas para un uso doméstico real.

Tabla 4. Resultados medios de Precision, Recall y F1-Score en las pruebas de movimiento frontal hacia la cámara.

Movimiento frontal hacia la cámara	Resultados Medios			
		25% Confianza	50% Confianza	75% Confianza
	Precision	0.87	0.87	0.92
	Recall	0.9	0.90	0.80
	F1-Score	0.44	0.44	0.43

Tabla 5. Resultados medios de Precision, Recall y F1-Score en las pruebas de movimiento de espaldas.

Movimiento de espaldas	Resultados Medios			
		25% Confianza	50% Confianza	75% Confianza
	Precision	0.75	0.82	0.90
	Recall	0.8	0.77	0.67
	F1-Score	0.39	0.39	0.38

Tabla 6. Resultados medios de Precision, Recall y F1-Score en las pruebas de presencia parcial del cuerpo.

Presencia parcial del cuerpo	Resultados Medios			
		25% Confianza	50% Confianza	75% Confianza
	Precision	0.66	0.71	0.76
	Recall	0.67	0.60	0.47
	F1-Score	0.33	0.32	0.29

Tabla 7. Resultados medios de Precision, Recall y F1-Score en las pruebas de luz diurna.

Luz diurna	Resultados Medios			
		25% Confianza	50% Confianza	75% Confianza
	Precision	0.80	0.87	0.92
	Recall	0.93	0.90	0.80
	F1-Score	0.43	0.44	0.43

Tabla 8. Resultados medios de Precision, Recall y F1-Score en las pruebas de luz nocturna.

Luz nocturna	Resultados Medios			
		25% Confianza	50% Confianza	75% Confianza
	Precision	0.63	0.66	0.72
	Recall	0.6	0.50	0.40
	F1-Score	0.31	0.28	0.25

La Ilustración 28 y la Ilustración 29 muestran dos ejemplos de detección durante las pruebas realizadas en un entorno real.

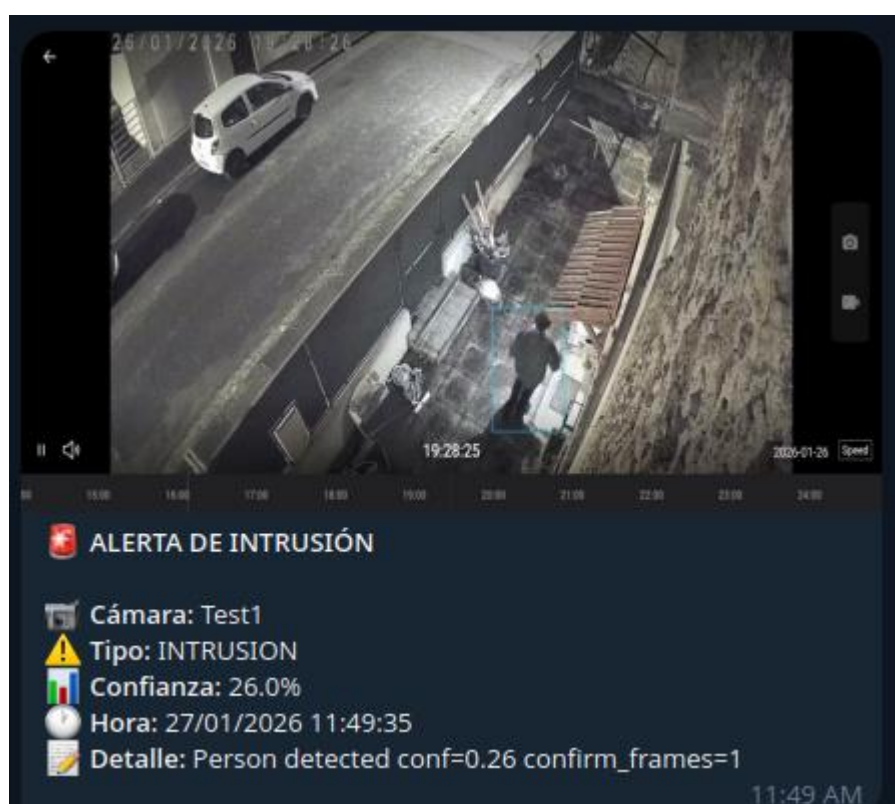


Ilustración 28. Ejemplo de detección de un intruso de espaldas con baja iluminación.



Ilustración 29. Ejemplo de detección de un intruso de frente a la cámara con iluminación diurna.

6.4.RENDIMIENTO COMPUTACIONAL

Los resultados recogidos en la Tabla 9 reflejan el impacto del número de cámaras activas sobre el rendimiento del sistema en un entorno local. Con una única cámara, el sistema mantiene una tasa elevada de procesamiento, alcanzando valores cercanos a 52 FPS con un uso moderado de CPU. Sin embargo, conforme aumenta el número de flujos simultáneos, se observa una degradación progresiva del rendimiento, reduciéndose los FPS y aumentando la latencia media debido a la carga computacional acumulada.

A partir de 4–5 cámaras, el procesador comienza a acercarse a la saturación, con consumos superiores al 80–90% y una caída significativa en la tasa de procesamiento. El consumo de memoria también crece de forma notable, alcanzando valores cercanos a 6 GB en el caso de 7 cámaras. Estos resultados confirman que el principal factor limitante en hardware embebido es la capacidad de cómputo disponible, aunque el sistema sigue siendo viable para un número reducido de cámaras.

Tabla 9. Métricas de rendimiento obtenidas durante la ejecución simultánea de múltiples cámaras en Raspberry Pi 5 (8 GB), incluyendo FPS medio, latencia y uso de CPU/RAM.

Num. Cámaras	Resolución de vídeo	FPS Medio	Latencia Media (ms)	Uso CPU (%)	Uso RAM (MB)
1	640x480	52	70	35.00	950
2	640x480	40	95	55.00	1200
3	640x480	28	130	70.00	1450
4	640x480	20	180	82.00	2220
5	640x480	14	240	90.00	3560
6	640x480	10	320	96.00	4100
7	640x480	4	500	99.00	6200

7. Conclusiones y trabajo futuro

7.1. CONCLUSIONES

El presente proyecto ha abordado el desarrollo de un sistema de videovigilancia doméstica inteligente centrado en dos aspectos clave: la privacidad del usuario y la viabilidad de ejecutar modelos de visión por computador de forma completamente local en hardware de recursos limitados. Frente a la creciente dependencia de soluciones comerciales basadas en suscripción y procesamiento en la nube, se ha propuesto una alternativa autónoma que permite mantener el control total de los datos dentro del entorno doméstico.

A lo largo del proyecto se ha diseñado e implementado una arquitectura modular capaz de capturar flujos de vídeo mediante RTSP, procesarlos en tiempo real y detectar la presencia de personas utilizando un modelo eficiente como MobileNet-SSD. Esta aproximación ha permitido reducir significativamente las falsas alarmas respecto a sistemas tradicionales basados únicamente en detección de movimiento, aportando un análisis semántico más fiable.

Se ha desarrollado un dashboard web completo basado en FastAPI que facilita la gestión del sistema, incluyendo el registro de usuarios, la administración de cámaras, la visualización del streaming en directo y la consulta de alertas generadas. Como complemento, se ha integrado un bot de Telegram que actúa como único canal externo de notificación, manteniendo la exposición del sistema al exterior bajo un enfoque controlado.

La evaluación experimental ha demostrado que el sistema alcanza un rendimiento satisfactorio en escenarios domésticos realistas, manteniendo niveles de precisión elevados en la detección de personas a distancias habituales y funcionando de manera estable en una Raspberry Pi 5. Los resultados de rendimiento computacional evidencian que el sistema es viable para un número reducido de cámaras simultáneas, lo que coincide con el uso esperado en viviendas particulares.

El trabajo realizado confirma que es posible construir una solución de videovigilancia inteligente y respetuosa con la privacidad mediante procesamiento local y modelos optimizados. La herramienta desarrollada constituye una base funcional sólida sobre la que

pueden incorporarse futuras mejoras orientadas a ampliar capacidades, incrementar eficiencia o integrar nuevas técnicas de análisis visual.

7.2.LÍNEAS DE TRABAJO FUTURO

A pesar de que el sistema desarrollado cumple satisfactoriamente los objetivos principales del proyecto, existen varias líneas de mejora que podrían ampliarse en trabajos posteriores para aumentar sus capacidades y su aplicabilidad.

En primer lugar, una evolución natural del sistema sería la incorporación de nuevos módulos de detección orientados a otras clases de interés, como vehículos, mascotas u objetos específicos. Esto permitiría enriquecer el análisis semántico del entorno y adaptar el sistema a distintos escenarios de vigilancia más allá de la detección de personas.

En segundo lugar, una extensión relevante consistiría en retomar la integración de detección y reconocimiento facial, lo que permitiría distinguir entre individuos autorizados y posibles intrusos de forma automática. Esta funcionalidad mejoraría la precisión del sistema y reduciría aún más la generación de alertas innecesarias en contextos domésticos.

Finalmente, también podrían explorarse mejoras relacionadas con la optimización del rendimiento en múltiples cámaras simultáneas y la integración de aceleradores hardware específicos para inferencia en dispositivos embebidos.

Referencias bibliográficas

- Advantage Technology. (4 de Agosto de 2025). *Comparing Traditional CCTV with AI-Powered Surveillance*. Obtenido de Advantage Tech: <https://www.advantage.tech/comparing-traditional-cctv-with-ai-powered-surveillance/>
- Ashish, V., Noam, S., Niki, P., Jakob, U., Llion, J., Aidan N, G., . . . Illia, P. (2017). Attention is All you Need. *Advances in Neural Information Processing Systems*, 30.
- BlueIris. (2026). *Video Security & Webcam Software*. Obtenido de Blue Iris: <https://blueirissoftware.com/>
- Bluenvirion. (Enero de 2026). *GitHub*. Obtenido de Mediamtx: <https://github.com/bluenvirion/mediamtx>
- Buchanan, B., & Smith, R. (1988). Fundamentals of expert systems. *Annual review of computer science*, 23-58.
- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, (págs. pp. 886-893 vol. 1). San Diego.
- Everingham, M., van Gool, L., Williams, C., Winn, J., & Zisserman, A. (2012). *The PASCAL Visual Object Classes Homepage*. Obtenido de Information Engineering at the University of Oxford: <https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/>
- FastAPI. (17 de Noviembre de 2025). *Templates Jinja*. Obtenido de FastAPI: <https://fastapi.tiangolo.com/advanced/templates/>
- FFmpeg. (Diciembre de 2025). *FFmpeg Documentation*. Obtenido de FFmpeg: <https://ffmpeg.org/documentation.html>
- GeeksForGeeks. (10 de Octubre de 2025). *SSR vs CSR vs SSG*. Obtenido de Geeks For Geeks: <https://www.geeksforgeeks.org/javascript/server-side-rendering-vs-client-side-rendering-vs-server-side-generation/>
- Girshick, R., Donahue, J. D., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. *Proceedings of the IEEE conference on computer vision and pattern recognition*, (págs. 580-587).

- Gobierno de España. (19 de Abril de 2023). *Plan de Recuperación, Transformación y Resiliencia*. Obtenido de Qué es la Inteligencia Artificial: <https://planderecuperacion.gob.es/noticias/que-es-inteligencia-artificial-ia-prtr>
- Howard, A. G., Menglong, Z., Bo, C., Dmitry, K., Weijun, W., Tobias, W., . . . Hartwig, A. (17 de Abril de 2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. Obtenido de Arxiv: <https://arxiv.org/abs/1704.04861>
- Jialin Pan, S., & Yang, Q. (16 de Octubre de 2009). *A Survey on Transfer Learning*. Obtenido de IEEE Explore: <https://ieeexplore.ieee.org/document/5288526>
- Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.
- LeCun, Y., Bengio, Y., & Hinton, G. (28 de Mayo de 2015). *Deep Learning*. Obtenido de Nature: <https://www.nature.com/articles/nature14539>
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C. Y., & Berg, A. C. (2016). Ssd: Single shot multibox detector. *European conference on computer vision* (págs. 21-37). Springer International Publishing.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (29 de Diciembre de 2016). *SSD: Single Shot MultiBox Detector*. Obtenido de Arxiv: <https://arxiv.org/abs/1512.02325>
- Loza, R. (s.f.). *Alarmas con cuota mensual vs sin cuota: Comparativa de rentabilidad a largo plazo*. Obtenido de quecomparo: <https://www.quecomparo.es/ahorro/seguridad/alarmas-con-cuota-mensual-vs-sin-cuota/>
- McCarthy, J., Minsky, M. L., Rochester, N., & Shannon, C. E. (2006). A proposal for the darmouth summer research project on artificial intelligence, august 31, 1955. *AI Magazine*, 12-24.
- Microsoft. (s.f.). *Visión por Computador*. Obtenido de Azure Microsoft: <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-computer-vision>

- Montes, S. (2025). Las alarmas residenciales mantienen su auge en España: crecen un 8% en el primer trimestre de 2025. *Escudo Digital*.
- Olamide, S. (31 de Octubre de 2025). *Hackernoon*. Obtenido de Building your first video pipeline: FFmpeg & MediaMTX basics: <https://hackernoon.com/part-1building-your-first-video-pipeline-ffmpeg-and-mediامتx-basics>
- ONTSI. (2023). *Uso de la inteligencia artificial y big data en las empresas españolas*. Obtenido de ONSI: https://www.ontsi.es/sites/ontsi/files/2023-02/Br%C3%BAjula_IA_Big_data_2023.pdf
- Ramírez, S. (23 de Diciembre de 2025). *FastAPI*. Obtenido de FastAPI: <https://fastapi.tiangolo.com/>
- Schulzrinne, H. e. (Diciembre de 2016). *RTSP RFC 7826*. Obtenido de rfc-editor.org: <https://www.rfc-editor.org/rfc/rfc7826>
- Schulzrinne, H. R. (Abril de 1998). *RTSP RFC 2326*. Obtenido de rfc-editor.org: <https://www.rfc-editor.org/rfc/rfc2326.html>
- Song, H., Huizi, M., & Dally, W. J. (2015). *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. Obtenido de <https://arxiv.org/abs/1510.00149>
- SQLite. (31 de Mayo de 2025). *Appropriate Uses for SQLite*. Obtenido de SQLite When To Use: <https://www.sqlite.org/whentouse.html>
- SQLite. (28 de Diciembre de 2025). *Documentation*. Obtenido de SQLite: <https://www.sqlite.org/docs.html>
- Weisong, S., Jie, C., Quan, Z., Youhuizi, L., & Lanyu, X. (2016). Edge Computing: Vision and Challenges. *IEEE internet of things journal*, pág. 637'646.
- Wikipedia. (16 de Abril de 2007). *Deep Blue versus Garry Kasparov*. Obtenido de Wikipedia: https://en.wikipedia.org/wiki/Deep_Blue_versus_Garry_Kasparov
- Wonza Media Systems. (Enero de 2025). *RTSP: The Real-Time Streaming Protocol Explained (Update)*. Obtenido de Wonza: <https://www.wonza.com/blog/rtsp-the-real-time-streaming-protocol-explained>

Yimyam, W., & Ketcham, M. (2018). Video Surveillance System Using IP Camera for Target Person Detection. *18th International Symposium on Communications and Information Technologies*, (págs. 176-179).

Anexo A. Código en Python para capturar el vídeo del protocolo RTSP

```
import cv2
import time

def build_gstreamer_pipeline(rtsp_url: str, latency_ms: int = 100) -> str:
    return (
        f"rtspsrc location={rtsp_url} latency={latency_ms} ! "
        "rtph264depay ! h264parse ! avdec_h264 ! "
        "videoconvert ! appsink drop=true sync=false"
    )

def open_rtsp(rtsp_url: str):
    cap = cv2.VideoCapture(rtsp_url, cv2.CAP_FFMPEG)
    if cap.isOpened():
        return cap, "FFMPEG"

    gst = build_gstreamer_pipeline(rtsp_url)
    cap = cv2.VideoCapture(gst, cv2.CAP_GSTREAMER)
    if cap.isOpened():
        return cap, "GSTREAMER"

    raise RuntimeError("No se pudo abrir el stream RTSP ni con FFMPEG ni con GStreamer.")

def main(rtsp_url: str, target_width: int = 640, reconnect_wait_s: float = 2.0):
    cap = None
    backend = None

    fps = 0.0
    prev_time = time.time()

    while True:
        if cap is None or not cap.isOpened():
            try:
                cap, backend = open_rtsp(rtsp_url)
                cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
                print(f"[OK] Conectado a RTSP usando backend: {backend}")
            except Exception as e:
                print(f"[WARN] No conectado. Reintentando en {reconnect_wait_s}s. Motivo: {e}")
                time.sleep(reconnect_wait_s)
                continue

        ok, frame = cap.read()
        if not ok or frame is None:
            print("[WARN] Frame inválido. Forzando reconexión")
```

Anexo B. Dockerfile de la aplicación completa

```
FROM python:3.13-slim as builder

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY app/requirements.txt /tmp/requirements.txt
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r /tmp/requirements.txt

FROM python:3.13-slim

RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1 \
    libglu1 \
    libjpeg62 \
    libpng16 \
    libsm6 \
    libxext6 \
    libxrender1 \
    libgl1 \
    libgstreamer1.0-0 \
    libgstreamer-plugins-base1.0-0 \
    ffmpeg \
    && rm -rf /var/lib/apt/lists/* \
    && apt-get clean

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

WORKDIR /app

COPY app/ ./app/
COPY models/ ./models/

RUN mkdir -p /app/evidences /app/data

ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
ENV DATA_DIR=/app/data
ENV EVIDENCES_DIR=/app/evidences
ENV MODELS_DIR=/app/models

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import urllib.request;
    urllib.request.urlopen('http://localhost:8000/health')" || exit 1
```

Anexo C. docker-compose de la aplicación completa

```
version: '3.8'

services:
  surveillance:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: surveillance-hq
    ports:
      - "8000:8000"
    volumes:
      - surveillance_data:/app/data
      - surveillance_evidences:/app/evidences
    environment:
      - SECRET_KEY=${SECRET_KEY:-change-this-secret-key-in-production}
      - TZ=Europe/Madrid
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s

volumes:
  surveillance_data:
    name: surveillance_data
  surveillance_evidences:
    name: surveillance_evidences
```

Anexo D. Repositorio GitHub

El código completo de la aplicación se encuentra publicado en el repositorio GitHub que se puede acceder con el siguiente enlace:

<https://github.com/Helguera/TFM-AI-UNIR-2026>